Tugas Besar I IF3170 Intelegensi Artifisial



Dipersiapkan oleh:

13524007 Muhammad Ashkar

13524076 Fayyaz Akmal Lauda

13524097 Muhammad Faiz Alfada Dharma

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026/2027

Daftar Isi

Daftar Isi **2**
Deskripsi Persoalan **3**
Pembahasan **4**
Representasi State dan Model Data **4**
State Pencarian dan Penyalinan Data 5
Perhitungan Ukuran dan Posisi Paket 6
Sel yang Ditempati Paket 7
Input Awal dan Pembentukan State 7
Pembentukan Grid dari Data Paket 8
Pembentukan Initial State Acak 8
Penggunaan State Awal oleh Setiap Algoritma 8
Program Utama dan Pemanggilan Algoritma 9
Penyajian Hasil Ketiga Algoritma 9
Objective Function dan Fitness Penalti 10
Komponen Penalti dan Validitas State 10
Perhitungan Berat dan Batas Ruang 11
Benturan dan Penyanggaan Paket 11
Algoritma Hill Climbing 12
Implementasi 12
Pembentukan Tetangga untuk Pencarian 13
Steepest Ascent 13
Hill Climbing dengan Sideways Move 13
Hill Climbing dengan Random Restart 14
Stochastic Hill Climbing 14
Penyajian Hasil Hill Climbing 14
Test Case 15
Analisis Hasil 15
Algoritma Simulated Anneling 15
Implementasi 15
Penerimaan Kandidat pada Simulated Annealing 15
Pencatatan Hasil Simulated Annealing 16
Penyajian Peluang Penerimaan Simulated Annealing 16
Test Case 16
Analisis Hasil 17
Algoritma Genetik 17
Implementasi 17
Bobot Seleksi dan Roulette Selection 17

Pemilihan Induk..... 18
Crossover Satu Titik.....19
Mutasi Posisi dan Orientasi.....20
Pembentukan Generasi Berikutnya.....21
Pengendalian Proses Genetic Algorithm.....22
Data Hasil dan Riwayat Populasi.....23
Penyajian Populasi Genetic Algorithm..... 25
Test Case..... 26
Rancangan Pengujian.....26
Analisis Hasil.....26
Kesimpulan dan Saran.....27
Pembagian Tugas Kelompok.....28
Format Deklarasi Penggunaan AI..... 29
Lampiran..... 31

Deskripsi Persoalan

Pada Tugas Besar 1 IF3170 Inteligensi Artifisial, persoalan yang diangkat adalah menentukan susunan paket yang optimal di dalam peti kemas sebuah truk milik perusahaan logistik Mejiro Express. Tujuan dari persoalan ini adalah memaksimalkan total nilai dari paket yang dapat diangkut oleh truk dengan tetap memperhatikan kapasitas dan ukuran *package*

Setiap *package* memiliki karakteristik yang berbeda, seperti ukuran, berat, nilai, orientasi, dan status barang pecah-belah (*fragile*). Sementara itu, *package* memiliki ukuran ruang dan batas berat maksimum. Oleh karena itu, tidak semua *package* harus dimasukkan ke dalam truk. Penempatan *package* juga harus memenuhi beberapa aturan, yaitu paket tidak boleh menembus dinding truk, tidak boleh melayang tanpa penopang, dan *package* pecah-belah tidak boleh menjadi penopang paket lain. Selain itu, total berat *package* tidak boleh melebihi kapasitas maksimum truk

Untuk mencari susunan paket yang optimal, pada tugas besar ini, digunakan pendekatan *local search* dengan tiga algoritma, yaitu *Hill Climbing* dan semua variansinya, *Simulated Annealing*, dan *Genetic algorithm*. Algoritma-algoritma tersebut digunakan untuk mencari susunan paket dengan nilai total setinggi mungkin melalui perubahan posisi, pertukaran, atau rotasi *package*

Hasil dari ketiga algoritma kemudian dibandingkan berdasarkan nilai akhir yang diperoleh, waktu pencarian, dan perkembangan nilai selama proses pencarian. Perbandingan ini dilakukan untuk mengetahui kelebihan dan kekurangan masing-masing algoritma dalam menyelesaikan persoalan optimasi penempatan paket

Pembahasan

Representasi State dan Model Data

Seluruh algoritma menggunakan objek Problem yang menyimpan truk dan daftar paket. Dimension menyimpan ukuran w, l, dan h. Truck menyimpan dimensi peti kemas dan max_capacity sebagai batas total berat. Kedua kelas memakai dataclass dengan frozen=True sehingga atributnya tidak diganti selama pencarian.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Dimension:
    w: int
    l: int
    h: int


from dataclasses import dataclass
from .dimension import Dimension

@dataclass(frozen=True)
class Truck:
    dimension: Dimension
    max_capacity: int
```

Package menyimpan atribut tetap dan atribut susunan. ID, dimension, value, weight, is_fragile, dan eta menjelaskan paket. Position dan orientation menyatakan keadaan paket pada suatu kandidat solusi. Ketika paket dibuat, position bernilai None dan orientasinya mengikuti sumbu awal.

```
@dataclass
class Package:
    id: str
    dimension: Dimension
    value: int
    weight: int
    is_fragile: bool
    eta: int
    position: Position | None
    orientation: Orientation
```

```
    def __init__(self, id: str, dimension: Dimension, value: int,
                 weight: int, is_fragile: bool, eta: int
        ):
        self.id = id
        self.dimension = dimension
        self.value = value
        self.weight = weight
        self.is_fragile = is_fragile
        self.eta = eta
        self.position = None
        self.orientation = Orientation("x", "y", "z")
```

State Pencarian dan Penyalinan Data

ProblemState menyimpan objek Problem dan grid tiga dimensi. Grid diakses dengan urutan grid[z][y][x]. Sumbu x menunjukkan lebar, y menunjukkan panjang, dan z menunjukkan tinggi. Setiap sel berisi himpunan ID paket sehingga lebih dari satu paket dalam sel yang sama dapat diketahui sebagai overlap.

```
@dataclass
class Problem:
    truck: Truck
    packages: list[Package]


@dataclass
class ProblemState:
    problem: Problem
    grid: list[list[list[set[str]]]]

    @property
    def truck(self) -> Truck:
        return self.problem.truck

    @property
    def packages(self) -> list[Package]:
        return self.problem.packages
```

Properti loaded_list dan unloaded_list memisahkan paket berdasarkan ada atau tidaknya posisi. Properti outside_volume menghitung bagian paket yang melewati batas truk. Fungsi clone membuat salinan mendalam sehingga perubahan kandidat

tidak mengubah state asal. Fungsi rebuild menyusun kembali grid dari posisi dan orientasi paket.

```
    def clone(self):
         return deepcopy(self)

    def rebuild(self):
         from ..utils.state_builder import build_state

         updated = build_state(self.problem)
         self.problem = updated.problem
         self.grid = updated.grid
```

Hill climbing dan simulated annealing membentuk tetangga dari salinan state melalui fungsi pada action.py. Genetic algorithm menyalin state induk, mengganti gen posisi atau orientasi, lalu membangun ulang grid. Ketiga cara tersebut mempertahankan hubungan antara daftar paket dan isi grid.

Perhitungan Ukuran dan Posisi Paket

Posisi paket merupakan sel acuan. Orientasi menunjukkan arah sisi w, l, dan h terhadap sumbu ruang. Jika suatu sisi mengarah ke sumbu negatif, sel acuan tidak selalu menjadi sudut terkecil balok. Karena itu, perhitungan ruang menggunakan oriented_offset yang menyimpan perpindahan berarah menuju sel ujung.

$$o = \varepsilon(d - 1), \quad \varepsilon \in \{-1, 1\} \quad (1)$$

$$a_{min} = min(a, a + o), \quad a_{max} = max(a, a + o) \quad (2)$$

$$d = |o| + 1 \quad (3)$$

Pada persamaan tersebut, d adalah panjang sisi dalam satuan sel, ε adalah arah sisi, a adalah koordinat sel acuan, dan o adalah offset. Perhitungan ini digunakan oleh pembentukan grid, pemeriksaan benturan, batas truk, dan penyanggaan pada semua algoritma.

```
@dataclass
class Orientation:
```

```
    w: Axis
    l: Axis
    h: Axis


@dataclass
class Position:
    x: int
    y: int
    z: int


    @property
    def oriented_offset(self):
        orientations = {
            'w': self.orientation.w,
            'l': self.orientation.l,
            'h': self.orientation.h,
        }

        dim = {}
        sizes = {
            'w': self.dimension.w,
            'l': self.dimension.l,
            'h': self.dimension.h,
        }

        for dimension, axis in orientations.items():
            sign = -1 if axis.startswith('-') else 1
            axis = axis.lstrip('-')

            dim[axis] = (sizes[dimension]-1) * sign

        return dim['x'], dim['y'], dim['z']
```

Sel yang Ditempati Paket

Fungsi dimensions_for_orientation memberikan ukuran pada sumbu x, y, dan z tanpa tanda arah. Ukuran ini digunakan saat mencari orientasi yang muat dan pada gambar paket. Rotasi dilakukan oleh rotate_x, rotate_y, dan rotate_z dengan pemetaan arah dari _rotate_axis.

Fungsi get_occupied_cells menentukan seluruh sel yang ditempati satu paket. Paket dengan posisi None menghasilkan himpunan kosong. Untuk paket yang dimuat,

ujung setiap sumbu dihitung dari posisi dan oriented_offset. Rentang sel memakai nilai terkecil sampai terbesar, termasuk kedua ujungnya.

Fungsi ini dipakai untuk memperbarui grid setelah move, swap, dan rotasi pada pencarian tetangga. Evaluator memakai batas koordinat yang sama untuk menilai susunan. Dengan demikian, arah negatif tetap menghasilkan ukuran ruang yang sesuai.

Input Awal dan Pembentukan State

Input dibaca dari JSON dengan schema_version bernilai satu. Data wajibnya adalah truck dan packages, sedangkan initial_state dapat disertakan untuk menentukan susunan tertentu. Loader memeriksa kelengkapan atribut, tipe data, keunikan ID, dimensi dan berat positif, serta orientasi yang membentuk rotasi. Kesalahan input dinyatakan melalui InputError.

Fungsi parse_problem menyusun objek Truck dan Package. Jika initial_state tersedia, read_placements mengisi posisi dan orientasi paket. Nilai null menandakan paket tidak dimuat. Fungsi load_problem membaca berkas, menolak field JSON yang berulang, dan meneruskan isinya ke parse_problem.

Susunan tetap dari input dapat diperiksa melalui build_state. Pada proses pencarian, pembentukan susunan acak dilakukan oleh generator initial state. Data awal pada berkas tidak otomatis dipertahankan ketika wrapper algoritma membuat state baru.

Pembentukan Grid dari Data Paket

Fungsi build_state memvalidasi dan menyalin data melalui copy_problem. Grid dibentuk menurut ukuran truk, lalu setiap paket yang dimuat ditempatkan berdasarkan batas koordinatnya. Hanya sel di dalam truk yang masuk ke grid. Bagian di luar truk tetap tercatat melalui posisi paket dan outside_volume. Batas max_cells mencegah alokasi grid yang terlalu besar.

Pembentukan Initial State Acak

Fungsi generate_initial_state dipakai sebagai dasar keadaan awal ketiga algoritma. Setiap paket dimuat dengan peluang load_probability yang secara default bernilai 0,5. Generator memilih orientasi yang muat, kemudian memilih posisi acak di dalam truk. Paket awal tidak keluar dari batas, tetapi belum tentu bebas benturan, kelebihan berat, atau masalah penyanggaan.

Penggunaan State Awal oleh Setiap Algoritma

Hill climbing memanggil generate_initial_state satu kali di dalam run_hillclimbing. Random restart membuat state baru pada setiap restart. Simulated annealing membentuk state baru pada setiap run di dalam run_simulated_annealing. Program utama juga dapat memanggil generator langsung sebelum search_simulated_annealing. Genetic algorithm menggunakan generate_population untuk membentuk sekumpulan state awal.

Algoritma	**Keadaan awal**	**Pembentukan kandidat**
Hill climbing	Satu ProblemState acak	Seluruh tetangga atau satu tetangga acak sesuai varian.
Simulated annealing	Satu ProblemState acak pada setiap run	Satu tetangga acak pada setiap iterasi.
Genetic algorithm	Populasi yang berisi beberapa ProblemState	Anak dari seleksi induk, crossover, dan mutasi.

Seed dapat diberikan pada generator awal agar susunan dapat diulang. HC dan SA memakai modul random untuk pemilihan tetangganya. GA memakai generator Random sendiri. Mengatur seed generator awal saja tidak otomatis mengatur seluruh keputusan acak HC dan SA.

Program Utama dan Pemanggilan Algoritma

Fungsi main pada main.py menyediakan pilihan algoritma dari terminal. Program meminta nama JSON, memilih algoritma, lalu mengulang pencarian sebanyak jumlah percobaan yang dimasukkan. Nilai jumlah percobaan ini berbeda dari batas iterasi di dalam HC, SA, atau jumlah generasi GA.

Pada versi ini, main.py mencari JSON di folder tests. Dataset yang tersimpan di repo berada di data/cases, data/examples, dan data/experiments. Berkas yang hendak dipakai melalui menu perlu ditempatkan pada lokasi yang dibaca program. Fungsi load_problem sendiri menerima path berkas sehingga tidak dibatasi pada folder tests.

HC dipanggil melalui run_hillclimbing. SA menggunakan generate_initial_state lalu search_simulated_annealing. GA dipanggil melalui run_ga dengan parameter defaultnya. Setelah pencarian selesai, visualize_result memilih fungsi gambar yang sesuai dan gambar disimpan ke folder output. Fungsi plt.show diganti sementara agar penyimpanan dapat berlangsung tanpa menunggu jendela ditutup.

Penyajian Hasil Ketiga Algoritma

Fungsi visualize_result memilih visualisasi berdasarkan nama algoritma. Create_figure membagi gambar menjadi dua tampilan state, satu area grafik, dan satu area informasi. Draw_state menggambar peti kemas serta paket yang dimuat dan tidak dimuat. Paket yang tidak dimuat diletakkan di samping truk dengan tepi merah. Draw_information menulis parameter dan ukuran proses pencarian.

Draw_state pada main saat ini memakai posisi sel acuan secara langsung untuk titik awal gambar. Hal ini perlu diperhatikan saat menyiapkan gambar hasil karena orientasi negatif memerlukan sudut minimum yang telah dijelaskan pada bagian perhitungan posisi. Perhitungan grid dan evaluator tetap memakai offset bertanda.

Objective Function dan Fitness Penalti

Evaluator digunakan bersama oleh hill climbing, simulated annealing, dan genetic algorithm. Nilai objective dihitung dari total value paket yang dimuat. Misalkan v_i adalah value paket dan b_i bernilai satu jika dimuat atau nol jika tidak dimuat. Total value dihitung sebagai berikut.

$$V(s) = i = 1nvibi \quad (4)$$

Skor pencarian disimpan pada state_value. Nilainya diperoleh dengan mengurangi total value dengan penalti. Dalam pembahasan ini, skor tersebut disebut fitness agar berbeda dari value muatan.

$$F(s) = V(s) - P(s) \quad (5)$$

Hill climbing membandingkan state_value untuk memilih langkah. Simulated anneling memakai selisih state_value pada keputusan penerimaan. Genetic algorithm memakai state_value untuk seleksi dan menentukan individu terbaik. Semua algoritma tetap dapat membaca value dan is_valid secara terpisah dari hasil evaluasi.

Komponen Penalti dan Validitas State

Penalti mempunyai dua bagian. Setiap jenis pelanggaran yang muncul menambah bobot 100. Besar pelanggarannya kemudian dikalikan bobot 10. Perhitungan ini berlaku sama untuk seluruh algoritma.

$$P(s) = 100K + 10(B + O + D + U + R) \quad (6)$$

Simbol	Komponen	Makna nilai
B	Overload	Kelebihan berat dibanding kapasitas truk.
O	Overlap	Jumlah volume irisan setiap pasangan paket.
D	Out of bounds	Jumlah sel paket di luar batas truk.
U	Unsupported	Jumlah paket tanpa dukungan pada alasnya.
R	Fragile support	Jumlah pasangan penyanggaan oleh paket fragile.
K	Jenis pelanggaran	Banyak komponen di atas yang nilainya lebih besar dari nol.

Evaluator menyimpan kelima komponen dalam violations dan menghitung berapa jenis yang aktif. State dinyatakan valid hanya jika semuanya nol. Bobot ini tidak menjamin bahwa kandidat valid selalu mengalahkan kandidat tidak valid, sehingga skor dan validitas tetap perlu dibaca bersama.

Angka-angka tersebut didapat dari seberapa pentingnya masing-masing pelanggaran terhadap hasil akhir. Semakin banyak jenis pelanggaran, maka semakin banyak langkah yang dibutuhkan untuk menghilangkan pelanggaran, karena itu jenis pelanggaran diberi nilai yang besar,

Perhitungan Berat dan Batas Ruang

Fungsi evaluate_load menjumlahkan berat paket yang dimuat. Jika total berat tidak melampaui kapasitas, komponen overload bernilai nol. Dengan q_i sebagai berat paket dan C sebagai kapasitas truk, perhitungannya adalah sebagai berikut.

$$W(s) = \sum_{i=1}^{n} q_i b_i, \quad B = max(0, W(s) - C) \quad (7)$$

Komponen out of bounds dihitung dari volume paket yang berada di luar truk. Properti outside_volume membandingkan volume total dengan volume yang beririsan dengan batas truk. Fungsi evaluate_out_of_bounds menjumlahkannya untuk seluruh paket yang dimuat.

$$D = \sum_{i\ dimuat} volume\ paket\ i - volume\ di\ dalam\ truk \quad (8)$$

Benturan dan Penyanggaan Paket

Fungsi evaluate_overlap memeriksa setiap pasangan paket. Panjang irisan pada setiap sumbu dihitung dari koordinat sel terkecil dan terbesar. Jika salah satu sumbu tidak beririsan, pasangan itu tidak menambah overlap. Karena koordinat mewakili sel, selisih batas ditambah satu.

$$L_x = max(0, min(x_{maks,1}, x_{maks,2}) - max(x_{min,1}, x_{min,2}) + 1) \quad (9)$$

$$O = \sum_{i<j} L_{x,ij} L_{y,ij} L_{z,ij} \quad (10)$$

Fungsi evaluate_unsupported memeriksa alas paket. Alas pada lantai langsung dianggap memiliki dukungan. Untuk paket yang lebih tinggi, sisi atas penyangga harus tepat satu sel di bawah alas dan mempunyai irisan pada arah x dan y.

Dukungan sebagian sudah cukup. Sentuhan tepi tanpa sel alas bersama tidak dihitung.

Fungsi evaluate_fragile_support memeriksa kontak yang sama, tetapi hanya untuk penyangga dengan is_fragile bernilai True. Jika paket fragile menopang paket lain, komponen fragile_support bertambah. Atribut ETA dibaca dan disimpan, tetapi evaluator pada main belum memasukkannya ke state_value.

Algoritma Hill Climbing

Implementasi

Hill climbing menelusuri satu state dan memilih perubahan berdasarkan fitness. Fungsi run_hillclimbing membentuk state awal, memilih varian, menjalankan pencarian, dan menambahkan durasi. Hasilnya memuat initial, current, all_states, dan iteration. Varian tertentu juga menambahkan jumlah sideways atau rincian restart.

Varian	Fungsi	Parameter pembatas
Steepest ascent	steepest_ascent	Berhenti saat tidak ada tetangga dengan skor lebih tinggi.
Sideways move	sideways_move	max_sideways_move dengan nilai default 10.
Random restart	random_restart	max_restart dengan nilai default 10.
Stochastic	stochastic	max_iteration dengan nilai default 100.

Pembentukan Tetangga untuk Pencarian

Fungsi generate_all_successor menyusun semua tetangga yang diperiksa steepest ascent dan sideways move. Tetangga diperoleh dengan memindahkan setiap paket ke setiap koordinat truk, menukar setiap pasangan paket, dan merotasi paket yang sudah dimuat. Paket yang belum dimuat dapat masuk melalui operasi move. Daftar ini juga dipakai oleh setiap pencarian steepest ascent pada random restart.

Stochastic hill climbing dan SA memakai generate_random_successor yang memilih satu operasi move, swap, atau rotasi. Fungsi move_package, swap_package, dan rotate_package bekerja pada salinan state, kemudian memperbarui sel lama dan sel baru. Kandidat tidak disaring berdasarkan validitas sebelum dievaluasi.

Steepest Ascent

Fungsi steepest_ascent mengevaluasi seluruh tetangga dan memilih yang fitnessnya paling tinggi. Pencarian berhenti jika daftar tetangga kosong atau skor terbaiknya tidak melebihi skor current. Jika lebih baik, current diganti dan nilainya ditambahkan ke all_states. Jumlah iteration bertambah untuk setiap perpindahan yang diterima.

Hill Climbing dengan Sideways Move

Fungsi sideways_move memilih tetangga terbaik seperti steepest ascent. Perbedaannya, tetangga dengan skor yang sama dapat diterima selama batas sideways belum tercapai. Count_sideways hanya bertambah ketika skor tetangga sama dengan current. Pada implementasi ini, penghitung tersebut bersifat kumulatif dan tidak direset setelah perbaikan.

Bagian awal fungsi membentuk current, penghitung, dan all_states. Sesudah seluruh tetangga dinilai, kode berikut menentukan apakah pencarian berhenti atau menerima tetangga. Skor yang lebih buruk menghentikan pencarian. Skor seimbang dihentikan jika count_sideways sudah mencapai max_sideways_move.

Nilai sideways_move pada hasil menunjukkan jumlah langkah seimbang yang telah diterima. Nilai max_sideways menyimpan batasnya agar dapat ditampilkan bersama hasil pencarian.

Hill Climbing dengan Random Restart

Fungsi random_restart menjalankan steepest ascent pada state awal, lalu membentuk state acak baru sebanyak max_restart. Setiap hasil dibandingkan menggunakan fitness. Pasangan state awal dan akhir dari pencarian terbaik disimpan sebagai hasil utama.

Nilai max_restart menunjukkan banyak pengulangan setelah pencarian pertama. Jadi, nilai 10 berarti ada satu pencarian awal dan sepuluh restart. Total iteration menjumlahkan perpindahan seluruh pencarian. Iteration_per_restart dan objective_per_restart menyimpan rincian setiap pencarian, sedangkan all_states tetap berasal dari pencarian yang terpilih sebagai hasil terbaik.

Stochastic Hill Climbing

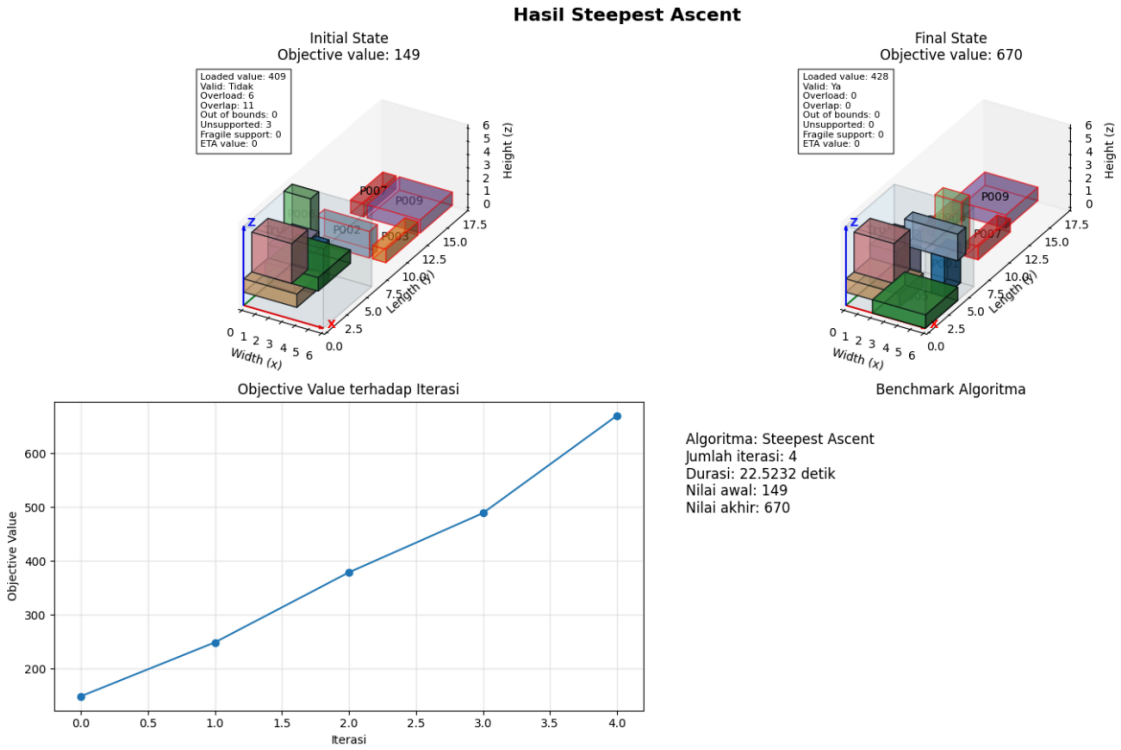
Fungsi stochastic mengambil satu tetangga acak pada setiap iterasi. Kandidat hanya diterima jika fitnessnya lebih tinggi. Jika ditolak, current tetap sama, tetapi nilainya tetap dicatat dalam all_states. Pencarian berhenti ketika count_iteration mencapai max_iteration. Karena nilai awal juga disimpan, jumlah catatan adalah satu lebih banyak daripada jumlah iterasi.

Penyajian Hasil Hill Climbing

Visualisasi HC membaca initial, current, all_states, iteration, dan duration. Sideways menambahkan jumlah serta batas sideways. Random restart membaca objective_per_restart dan iteration_per_restart. Nilai all_states berasal dari state_value, sehingga grafik merupakan fitness setelah penalti meskipun sebagian label program masih menyebutnya objective value.

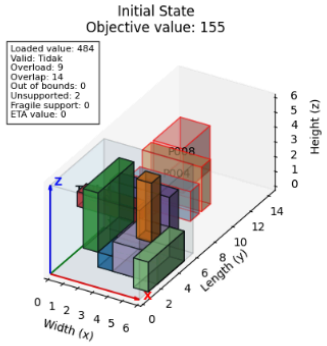
Pada main saat ini, visualize_random_restart memakai start_iteration sebelum variabel tersebut diisi. Bagian itu perlu dilengkapi sebelum gambar random restart dibuat. Rincian restart tetap tersedia pada hasil pencarian.

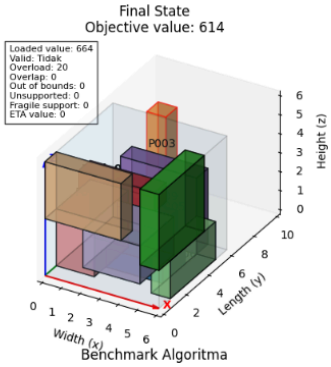
Test Case 1 – Hill Climbing Steepest Ascent

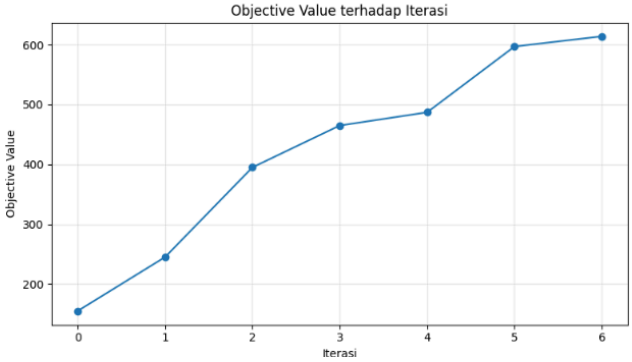


Algoritma: Steepest Ascent
Jumlah iterasi: 4
Durasi: 22.5232 detik
Nilai awal: 149
Nilai akhir: 670

Hasil Steepest Ascent

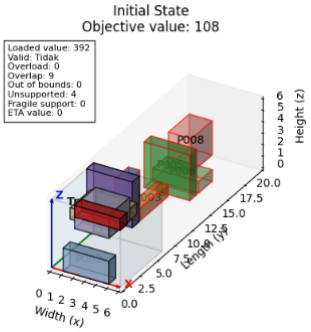


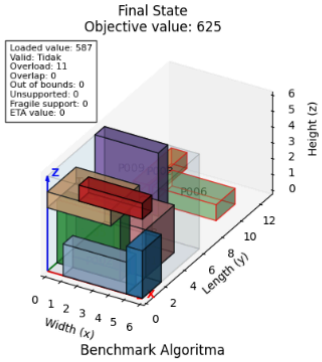


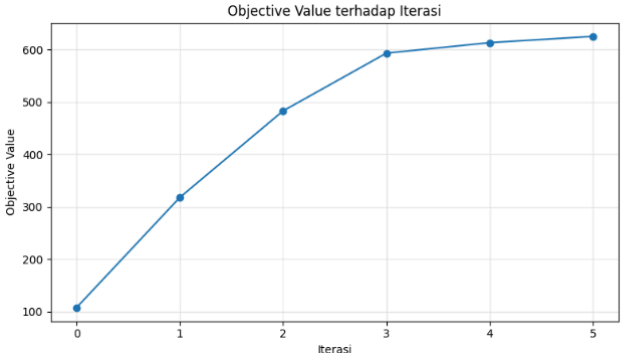


Algoritma: Steepest Ascent
Jumlah iterasi: 6
Durasi: 32.7348 detik
Nilai awal: 155
Nilai akhir: 614

Hasil Steepest Ascent

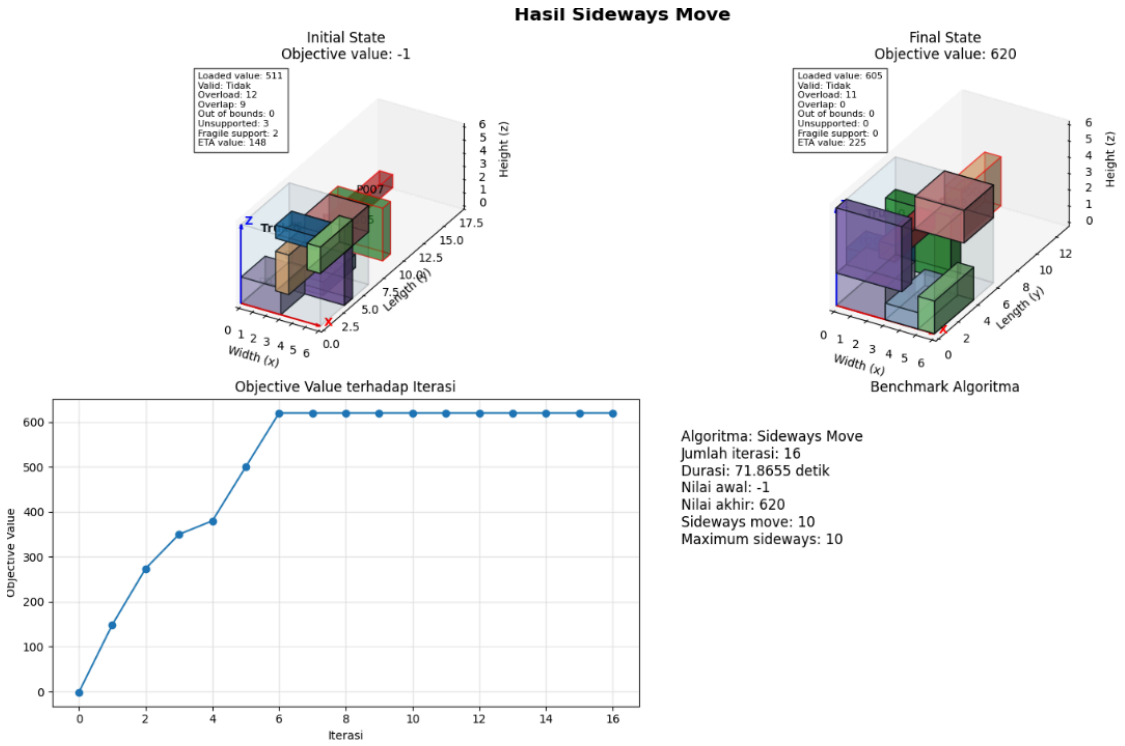




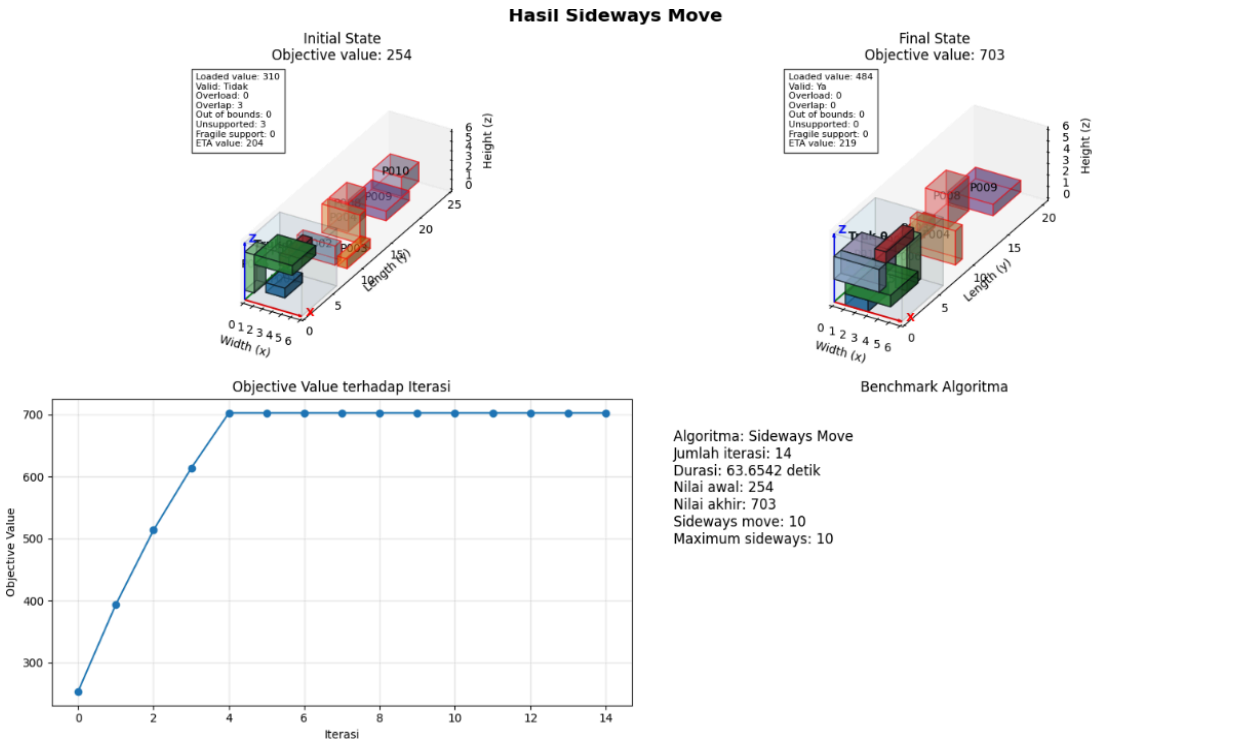


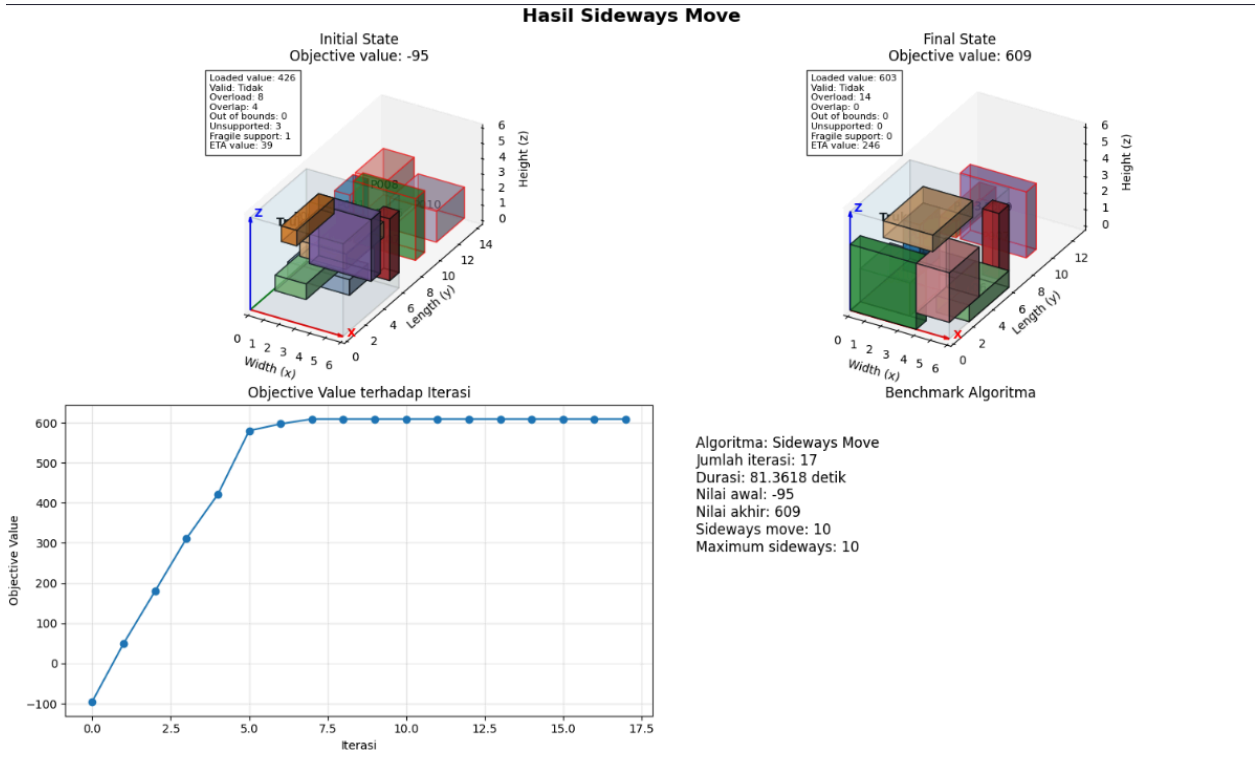
Algoritma: Steepest Ascent
Jumlah iterasi: 5
Durasi: 24.0034 detik
Nilai awal: 108
Nilai akhir: 625

Test Case 2 – Hill Climbing Sideways Move



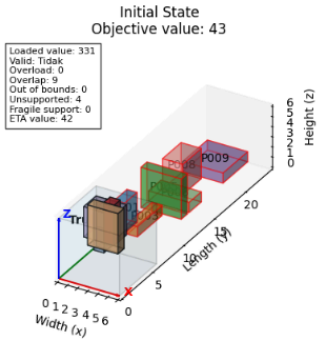
Algoritma: Sideways Move
Jumlah iterasi: 16
Durasi: 71.8655 detik
Nilai awal: -1
Nilai akhir: 620
Sideways move: 10
Maximum sideways: 10

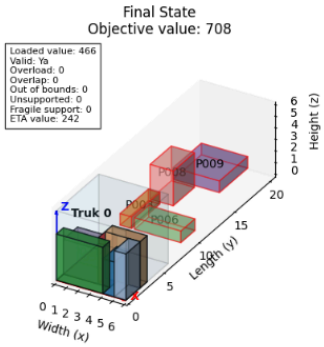


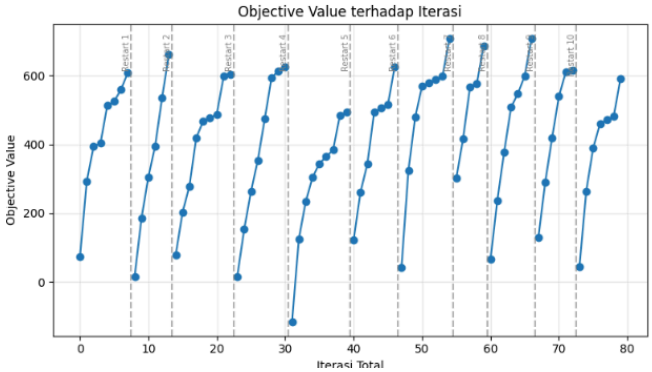


Test Case 3 – Hill Climbing Random Restart

Hasil Random Restart



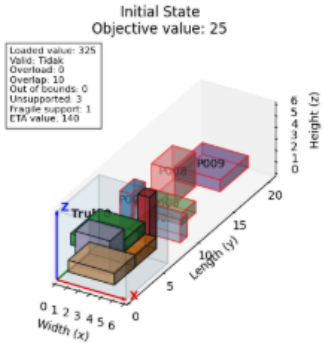


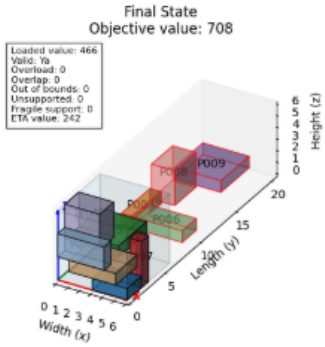


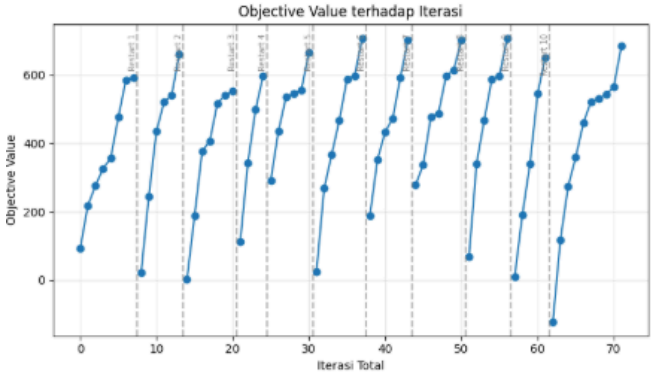
Benchmark Algoritma

Algoritma: Random Restart
Jumlah iterasi: 69
Durasi: 349.9636 detik
Nilai awal: 43
Nilai akhir: 708
Jumlah restart: 10
Maximum restart: 10
Iterasi per restart: [7, 5, 8, 7, 8, 6, 7, 4, 6, 5, 6]

Hasil Random Restart



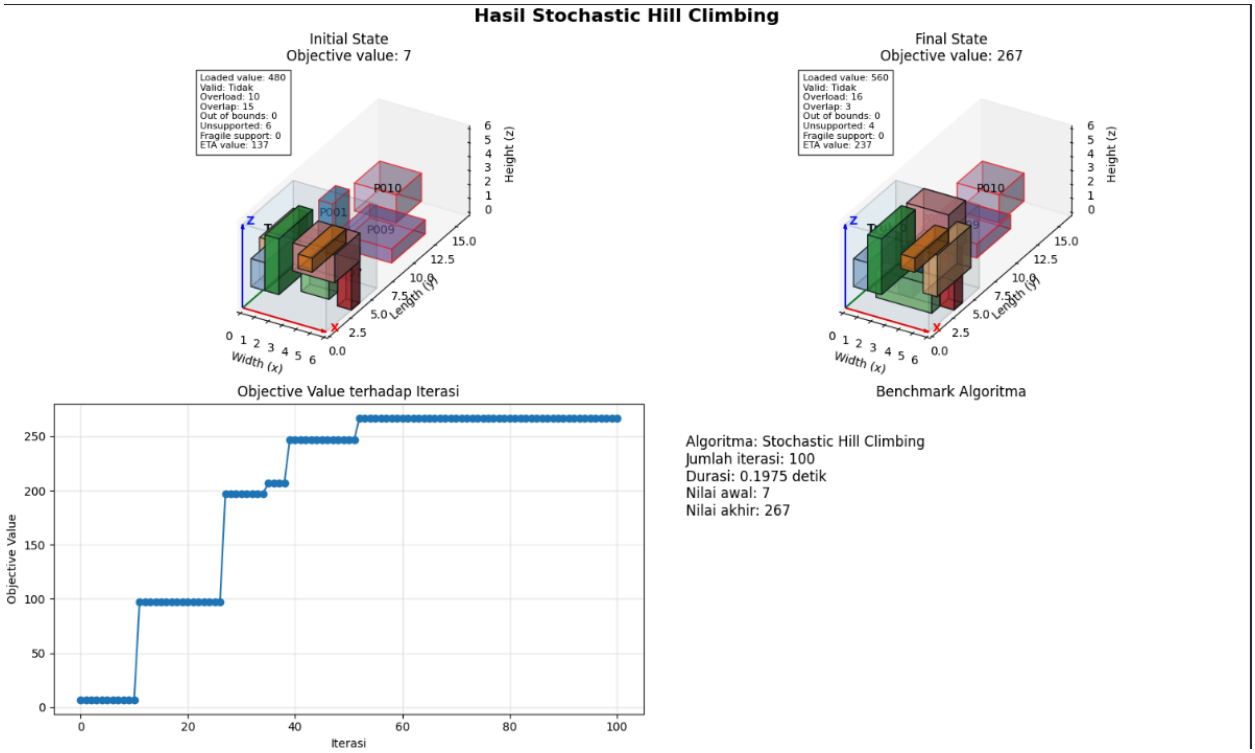


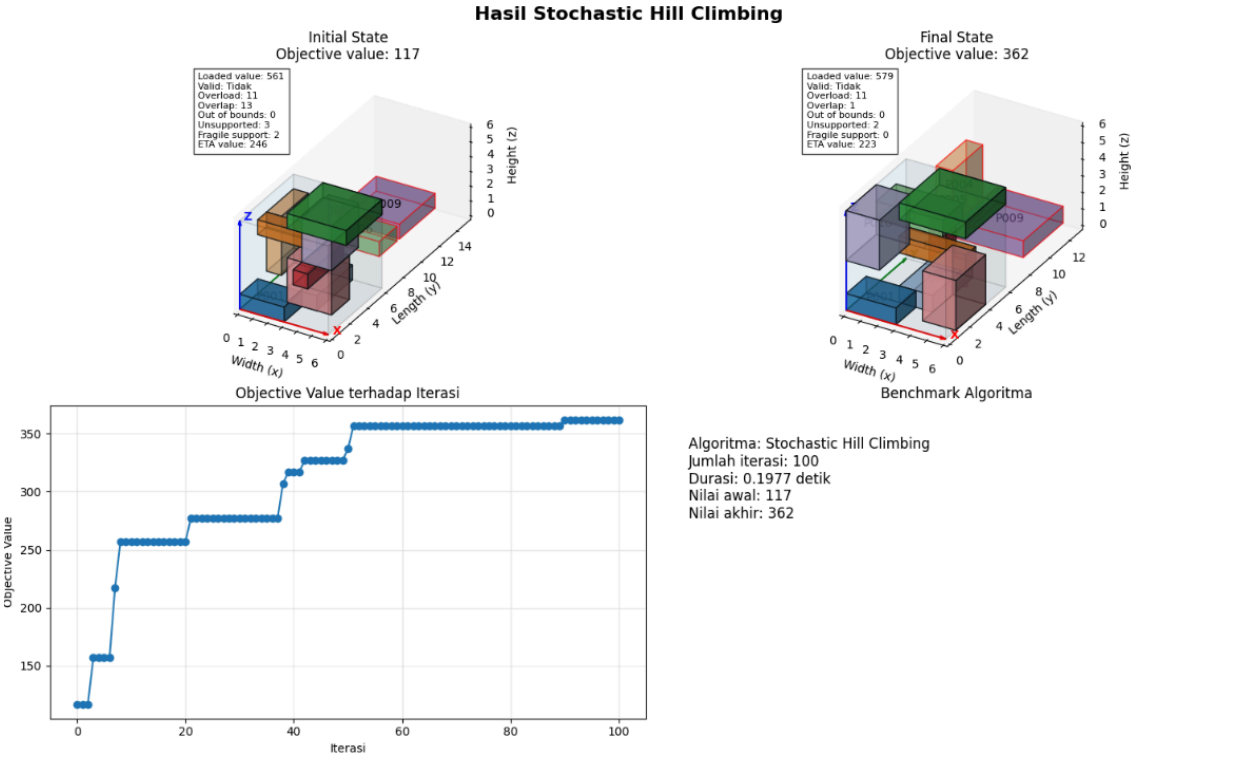


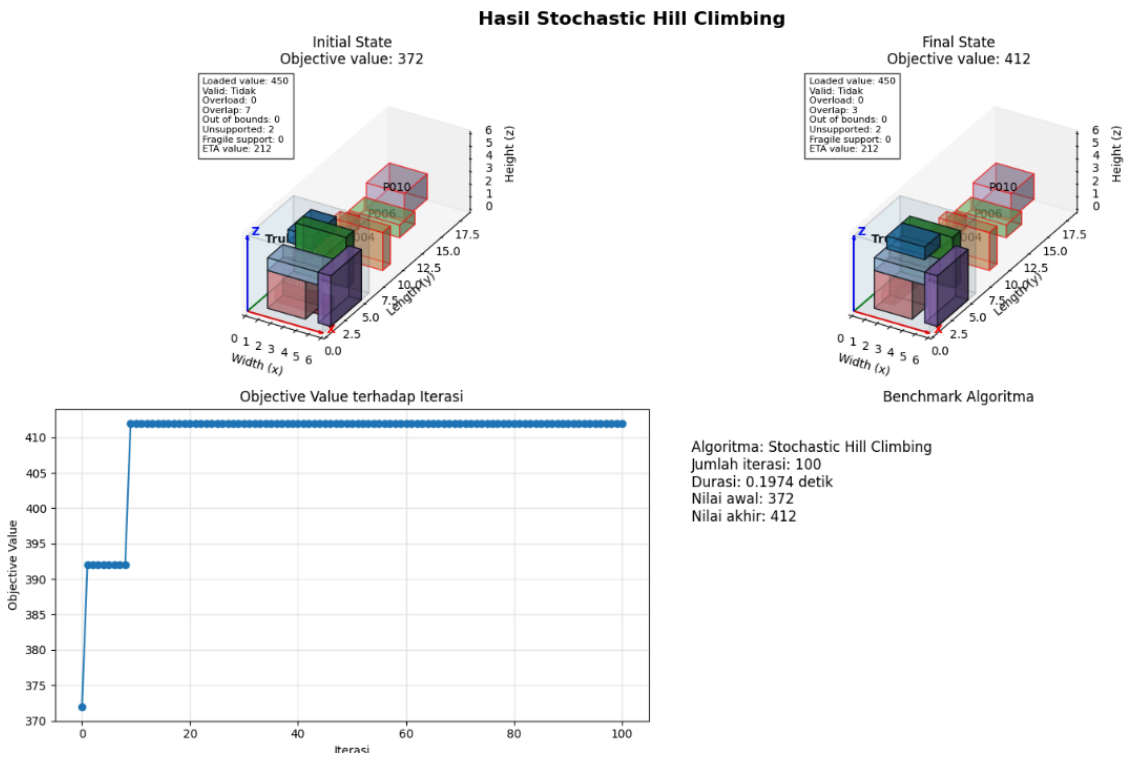
Benchmark Algoritma

Algoritma: Random Restart
Jumlah iterasi: 61
Durasi: 305.2117 detik
Nilai awal: 25
Nilai akhir: 708
Jumlah restart: 10
Maximum restart: 10
Iterasi per restart: [7, 5, 6, 3, 5, 6, 5, 6, 5, 4, 9]

Test Case 4 – Hill Climbing Stochastic







Analisis Hasil

Global optimum pada testcase ini belum diketahui, tetapi nilai terbaik eksperimen adalah 708 dari Random Restart, disusul Sideways Move 703, Steepest Ascent 670, dan Stochastic 412. Random Restart paling mendekati solusi terbaik karena menjalankan Steepest Ascent dari banyak initial state sehingga lebih mampu keluar dari solusi maksimum lokal. Sideways Move juga cukup baik karena tetap dapat melanjutkan pencarian ketika beberapa solusi tetangga memiliki nilai yang sama, sedangkan Steepest Ascent langsung berhenti jika tidak menemukan nilai yang lebih tinggi. Stochastic menghasilkan nilai terendah karena hanya memeriksa satu solusi tetangga secara acak pada setiap iterasi.

Dari segi durasi, Stochastic paling cepat dengan rata-rata 0,20 detik, dilanjutkan Steepest Ascent 26,42 detik, Sideways Move 72,29 detik, dan Random Restart 349,96 detik. Hasil Steepest Ascent berada pada rentang 614–670, Sideways Move 609–703, dan Stochastic 267–412, sehingga hasilnya masih dipengaruhi initial state; hanya satu dari tiga percobaan Steepest Ascent dan Sideways Move yang valid, sedangkan seluruh hasil Stochastic tidak valid.

Algoritma Simulated Annealing

Implementasi

Simulated annealing menelusuri satu state dan dapat menerima kandidat yang lebih buruk. Fungsi run_simulated_annealing mengulang pencarian sebanyak iteration_amount. Nilai tersebut adalah jumlah run, bukan jumlah langkah di dalam satu pencarian. Setiap run memakai initial state baru dan menghasilkan satu dictionary hasil.

```
def run_simulated_annealing(problem: Problem, iteration_amount:
int):
    results = []

    for i in range(iteration_amount):
        init_state = generate_initial_state(problem)
        results.append(search_simulated_annealing(init_state))

    return results
```

Fungsi search_simulated_annealing menjalankan proses pencarian. Temperatur awalnya 100 dan turun menjadi 0.99 nilai semula setiap iterasi. Selama temperatur masih diatas temperatur minimum yaitu 0.1, program mengambil satu tetangga melalui generate_random_successor. Jumlah iterasi pencarian normal pada pengaturan ini adalah 1000.

$$T_0 = 100, \quad T_{k+1} = T_k * 0.99 \quad (11)$$

Kedua state dinilai dengan evaluate_state yang sama seperti algoritma lain. Program membandingkan state_value sehingga perubahan skor mencakup value muatan dan penalti.

Penerimaan Kandidat pada Simulated Annealing

Kandidat yang lebih baik langsung diterima. Untuk kandidat yang tidak lebih baik, program menghitung peluang penerimaan dari selisih skor dan temperatur. Bilangan acak dari nol sampai kurang dari satu dibandingkan dengan peluang tersebut. Kandidat dengan skor sama memiliki peluang satu sehingga selalu diterima pada temperatur positif.

$$\Delta F = F(tetangga) - F(current) \quad (12)$$

$$p = e^{\Delta F/T}, \quad \Delta F \leq 0, \quad T > 0 \quad (13)$$

```
def _count_probability(state_value: int, successor_value: int, t:
int) -> float:
    d_e = successor_value - state_value

    return math.exp(d_e/t)
```

Bagian loop berikut memperlihatkan keputusan penerimaan, pencatatan state, dan penurunan temperatur. Plot_objective mencatat state_value setelah keputusan pada setiap iterasi. Plot_probability mencatat peluang kandidat yang dipertimbangkan, termasuk nilai satu ketika kandidat lebih baik.

```
while t > TEMPERATURE_MINIMUM:
        successor = generate_random_successor(state)

        successor_value = evaluate_state(successor,
consider_eta)["state_value"]

        # Tentukan apakah successor akan dipilih
        choose_successor: bool = False
        probability = 1.0

        if successor_value > state_value:
            choose_successor = True
        else:
            probability = _count_probability(state_value,
successor_value, t)
            random_value = random.random()
            if probability > random_value:
                choose_successor = True

        if choose_successor:
            state = successor
            state_value = successor_value

            if state_value > best_value:
                best_state = state
                best_value = state_value
        else:
            stuck += 1

        iteration += 1
        plot_objective[iteration] = state_value
```

```
        if probability == 1:
            plot_probability[iteration] = 0
        else:
            plot_probability[iteration] = probability
        t *= COOLING_RATE
```

Pencatatan Hasil Simulated Annealing

Penghitung stuck bertambah ketika kandidat ditolak. Jadi, stuck pada kode ini berarti jumlah penolakan kandidat selama run, bukan pengukuran langsung berapa kali algoritma mencapai optimum lokal. Current tetap dipakai untuk iterasi berikutnya jika kandidat ditolak. Jika diterima, current diganti kandidat tersebut.

```
result = {
        "initial": initial_state,
        "current": best_state,
        "best_value": best_value,
        "all_states": list(plot_objective.values()),
        "probability_history": list(plot_probability.values()),
        "iteration": iteration,
        "duration": perf_counter() - start_time,
        "stuck": stuck,
    }

return result
```

Dictionary hasil menyimpan state awal dan akhir, history skor, history peluang penerimaan, banyak iterasi, durasi, dan stuck. All_states dimulai dari nilai state awal, sedangkan probability_history dimulai dari keputusan iterasi pertama. Karena itu, keduanya memiliki panjang yang berbeda satu catatan.

Penyajian Peluang Penerimaan Simulated Annealing

Fungsi visualize_simulated_annealing menampilkan state awal dan akhir, history state_value, serta probability_history. Peluang penerimaan ditampilkan pada sumbu vertikal kedua. Sumbu iterasinya dimulai dari satu karena keputusan penerimaan pertama terjadi setelah state awal.

```
   graph_ax.plot(history, color="tab:blue", label="Objective
value")
    graph_ax.set(title="Objective dan Probability terhadap
Iterasi", xlabel="Iterasi", ylabel="Objective Value")
    graph_ax.grid(alpha=0.3)

    probability_ax = graph_ax.twinx()
    probability_ax.plot(range(1, len(probability_history) + 1),
probability_history, color="orange", label="Probability")
    probability_ax.set_ylabel("Probability")

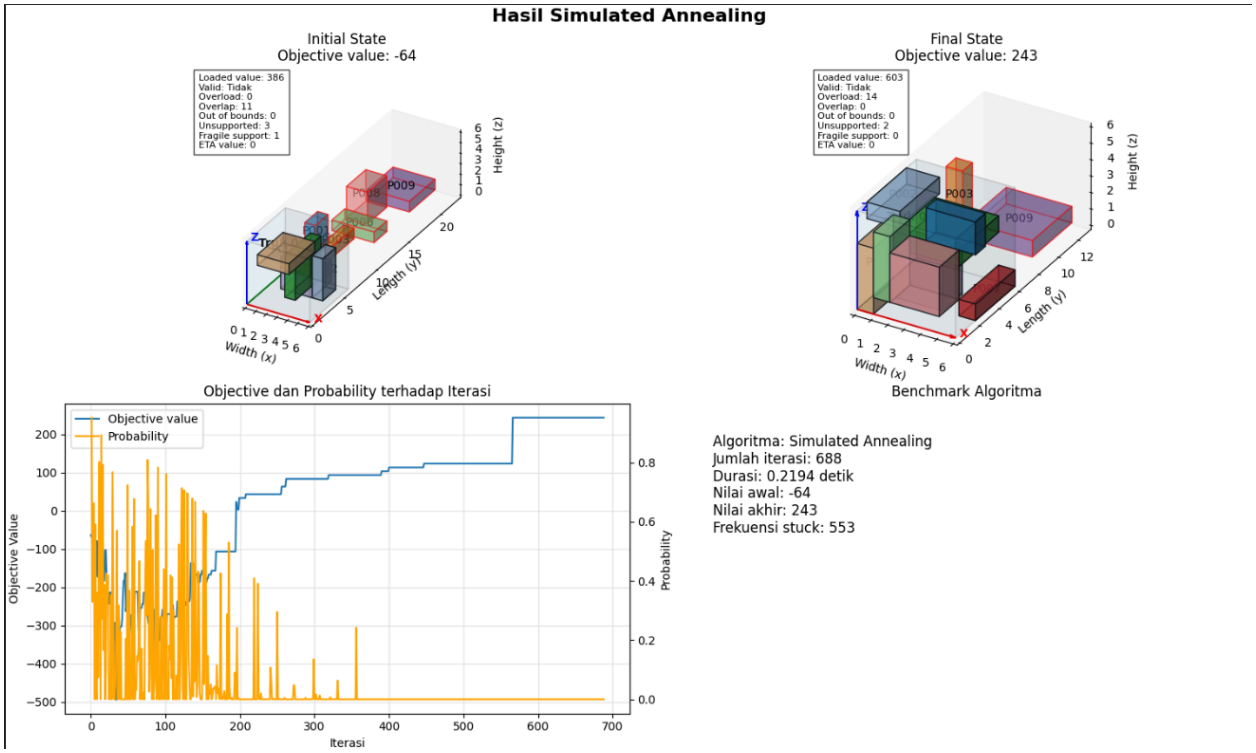
    graph_lines = graph_ax.get_lines() + probability_ax.get_lines()
    graph_labels = []

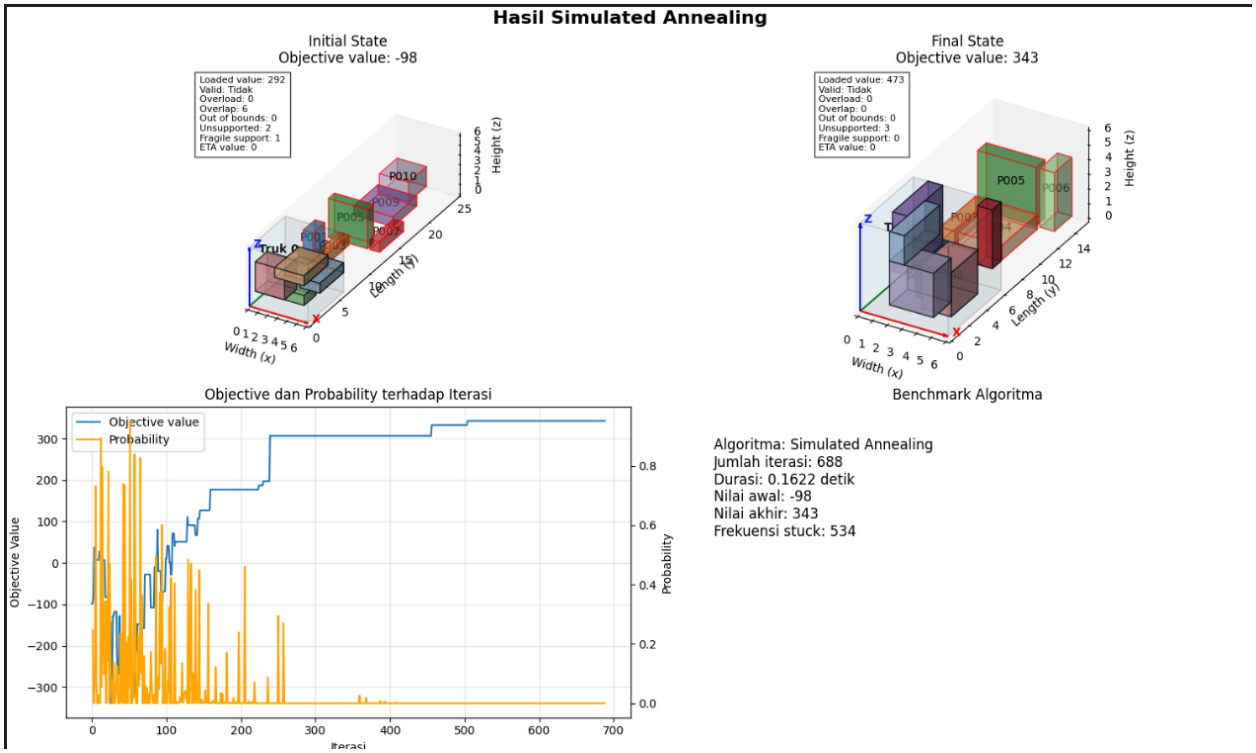
    for line in graph_lines:
        graph_labels.append(line.get_label())

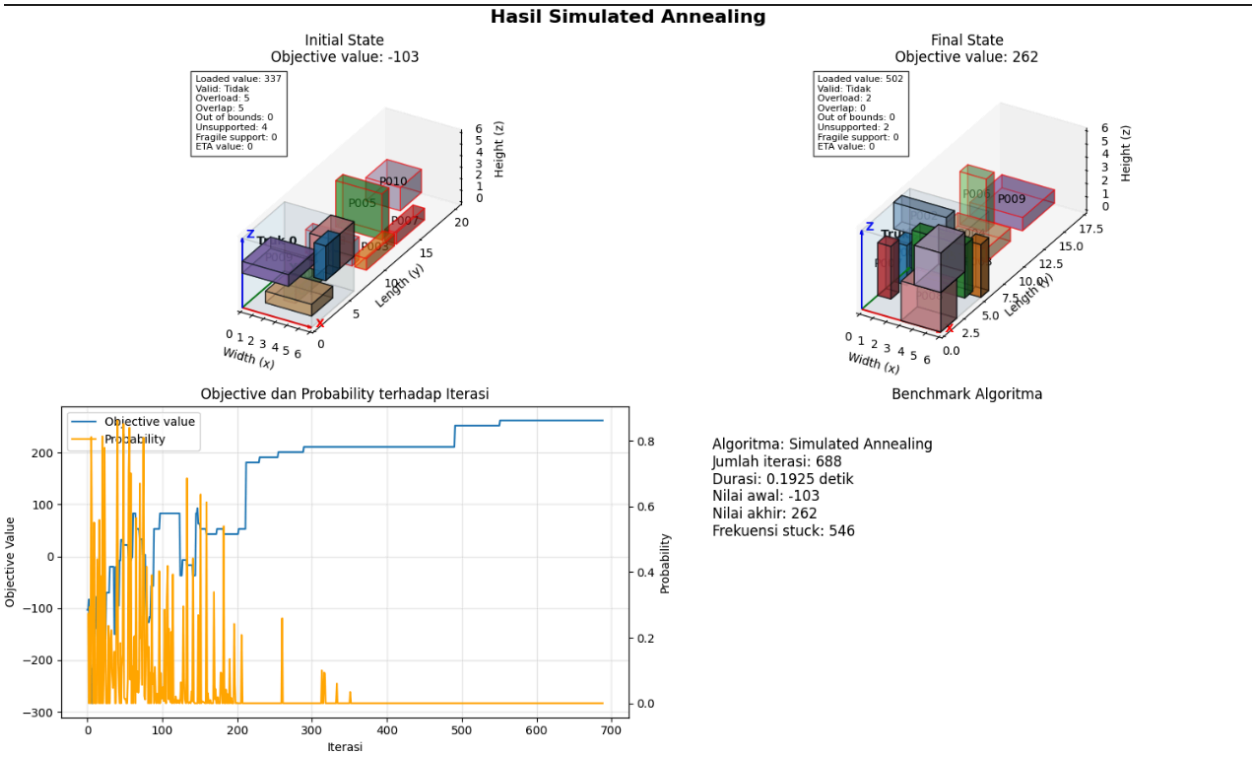
    graph_ax.legend(graph_lines, graph_labels)
```

Informasi hasil SA mencakup banyak iterasi, durasi, nilai awal dan akhir, serta stuck. Angka pada history merupakan fitness, sedangkan probability_history merupakan peluang kandidat yang dipertimbangkan. Kedua kurva memakai skala berbeda dan tidak menggambarkan besaran yang sama.

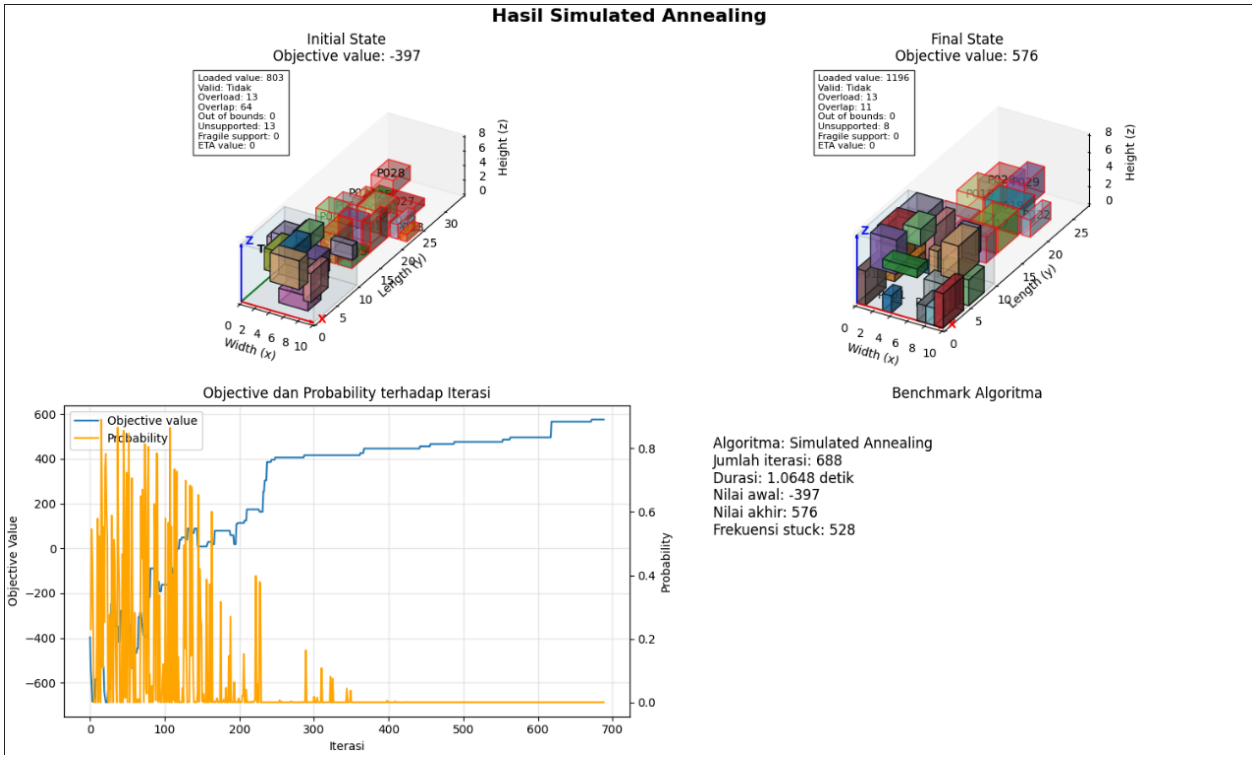
Test Case 1 – Simulated Annealing



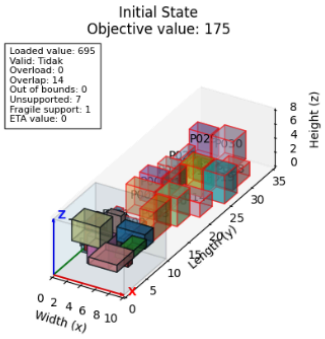


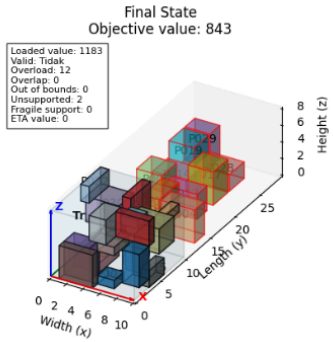


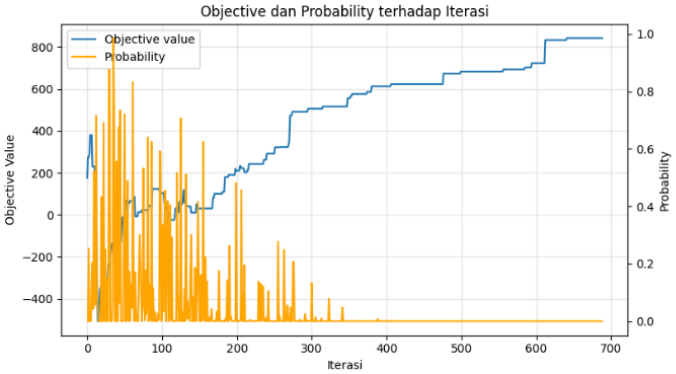
Test Case 2 – Simulated Annealing



Hasil Simulated Annealing



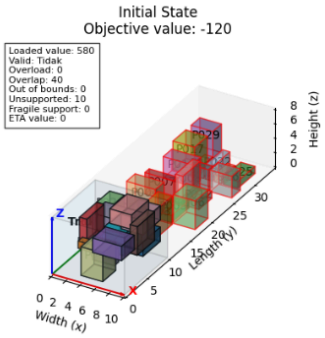


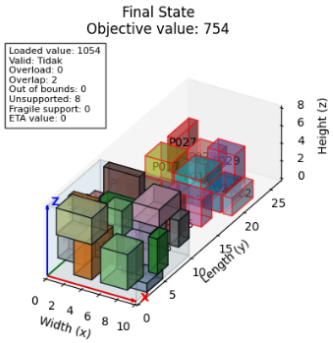


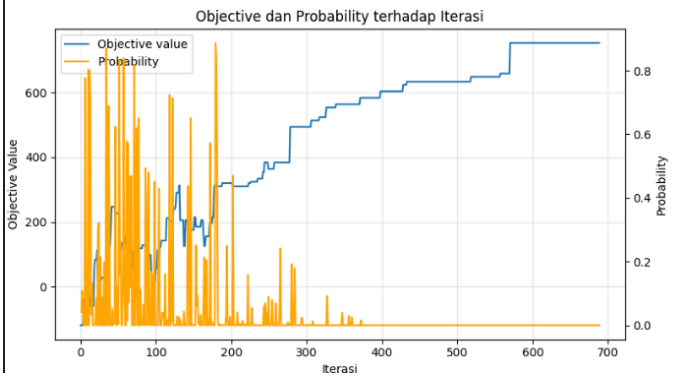
Benchmark Algoritma

Algoritma: Simulated Annealing
Jumlah iterasi: 688
Durasi: 0.9055 detik
Nilai awal: 175
Nilai akhir: 843
Frekuensi stuck: 506

Hasil Simulated Annealing







Benchmark Algoritma

Algoritma: Simulated Annealing
Jumlah iterasi: 688
Durasi: 0.8233 detik
Nilai awal: -120
Nilai akhir: 754
Frekuensi stuck: 511

Analisis Hasil

Pada kedua test case yang masing-masing dijalankan sebanyak 3 kali, bisa dilihat bahwa algoritma simulated annealing mencoba memilih successor yang state valuenya lebih kecil di awal, karena temperature di awal masih tinggi, sehingga kemungkinan untuk mencobanya masih besar. Semakin banyak jumlah iterasinya, semakin kecil kemungkinan untuk mencoba gagalnya. Bisa dilihat dari grafik state value yang terus naik dan grafik probability yang terus turun.

Frekuensi stuck adalah berapa kali algoritma mengalami 'stuck' selama algoritma dijalankan. Stuck pada program kami adalah kondisi ketika algoritma tidak memilih successornya karena probabilitynya terlalu kecil.

Algoritma Genetik

Implementasi

Genetic algorithm mengolah populasi state. Satu individu merupakan satu susunan seluruh paket. Gen pada urutan tertentu menyimpan posisi dan orientasi paket dengan urutan yang sama. Atribut tetap paket tidak berubah selama reproduksi. Package dengan position=None tetap menjadi bagian individu, tetapi tidak dihitung sebagai muatan.

Fungsi evaluate_population memasangkan state dengan hasil evaluator. Fungsi find_best_individual memilih fitness tertinggi. Jika terdapat skor yang sama, individu yang ditemukan lebih dahulu dipertahankan. Hasil evaluasi disimpan agar dapat digunakan pada seleksi dan pencatatan populasi.

Bobot Seleksi dan Roulette Selection

Fitness dapat bernilai negatif akibat penalti. Fungsi selection_weights mencari nilai terkecil, kemudian menggeser seluruh skor jika nilai tersebut negatif. Jika semua bobot nol, roulette_selection memilih individu secara merata. Jika jumlah bobot positif, peluang setiap individu mengikuti perbandingan bobotnya terhadap jumlah bobot populasi.

$$m = min(F_1, \ldots, F_N) \quad (14)$$

$$w_i = F_i - m + 1 \, untuk \, m < 0 \quad (15)$$

$$w_i = F_i \, untuk \, m \geq 0, \quad p_i = \frac{w_i}{\sum_{j=1}^{N} w_j} \quad (16)$$

```
def selection_weights(evaluated_population):
    if not evaluated_population:
        raise ValueError("Populasi tidak boleh kosong")

    minimum = evaluated_population[0]["evaluation"]["state_value"]

    for individual in evaluated_population:
        fitness = individual["evaluation"]["state_value"]
        if fitness < minimum:
            minimum = fitness

    weights = []

    for individual in evaluated_population:
        fitness = individual["evaluation"]["state_value"]
        if minimum < 0:
            weight = fitness - minimum + 1
        else:
            weight = fitness
        weights.append(weight)

    return weights
```

Pemilihan Induk

Fungsi roulette_selection membangkitkan angka acak antara nol dan jumlah bobot. Program menelusuri jumlah bobot kumulatif sampai melewati angka tersebut. Individu pada posisi itu dipilih menjadi induk. Pemilihan dilakukan dengan pengembalian sehingga individu yang sama dapat terpilih dua kali. Pergeseran bobot hanya memengaruhi peluang pemilihan, sedangkan fitness asli tetap disimpan.

```
def roulette_selection(evaluated_population, rng):
    weights = selection_weights(evaluated_population)
    total_weight = sum(weights)

    if total_weight == 0:
```

```
        return rng.choice(evaluated_population)

    target = rng.random() * total_weight
    cumulative = 0

    for index in range(len(evaluated_population)):
        cumulative += weights[index]
        if target < cumulative:
            return evaluated_population[index]

    for index in range(len(weights) - 1, -1, -1):
        if weights[index] > 0:
            return evaluated_population[index]
```

Fungsi check_parents memeriksa kesamaan truk, jumlah paket, urutan ID, dan atribut tetap kedua induk sebelum crossover. Pemeriksaan ini mencegah gen satu paket dicampurkan dengan data paket lain.

Crossover Satu Titik

Fungsi crossover mengambil bagian awal gen dari induk pertama dan bagian berikutnya dari induk kedua. Titik potong dipilih secara acak di antara gen pertama dan terakhir. Setiap pemanggilan menghasilkan satu anak. Jika jumlah paket kurang dari dua, program menyalin salah satu induk.

Posisi dan orientasi disalin dengan deepcopy agar perubahan pada anak tidak mengubah induknya. Setelah semua gen dipilih, rebuild menyusun grid anak. Crossover tidak memaksa anak menjadi valid sehingga hasilnya tetap dinilai dengan penalti.

```
def crossover(parent_1, parent_2, rng):
    state_1 = parent_1["state"]
    state_2 = parent_2["state"]

    check_parents(state_1, state_2)
    package_count = len(state_1.packages)

    if package_count < 2:
        return rng.choice([state_1, state_2]).clone()

    cut = rng.randint(1, package_count - 1)
    child = state_1.clone()
```

```
    for index in range(package_count):
        if index < cut:
            source = state_1.packages[index]

        else:
            source = state_2.packages[index]

        child.packages[index].position = deepcopy(source.position)
        child.packages[index].orientation =
deepcopy(source.orientation)

    child.rebuild()

    return child
```

Mutasi Posisi dan Orientasi

Fungsi mutate menyalin anak dan menentukan apakah mutasi dilakukan berdasarkan mutation_rate. Peluang defaultnya 0,2 per anak. Jika mutasi terjadi, satu paket dipilih secara acak. Paket yang belum dimuat dapat dimasukkan. Paket yang sudah dimuat dapat digeser satu sel, diputar 90 derajat, atau dikeluarkan. Susunan hasilnya kemudian dibangun ulang.

```
def mutate(child_state, mutation_rate, rng):
    if type(mutation_rate) not in (int, float):
        raise ValueError("Mutation rate harus berupa angka antara 0
dan 1")

    if not 0 <= mutation_rate <= 1:
        raise ValueError("Mutation rate harus berada antara 0 dan
1")

    mutated = child_state.clone()

    if not mutated.packages:
        return mutated

    if rng.random() >= mutation_rate:
        return mutated

    package = rng.choice(mutated.packages)

    if package.position is None:
        package.orientation = deepcopy(rng.choice(ROTATIONS))
        dimension = mutated.truck.dimension
```

```
        package.position = Position(rng.randint(0, dimension.w -
1), rng.randint(0, dimension.l - 1), rng.randint(0, dimension.h -
1))

    else:
        operation = rng.choice(["move", "rotate", "remove"])

        if operation == "move":
            axis = rng.choice(["x", "y", "z"])
            distance = rng.choice([-1, 1])
            if axis == "x":
                package.position.x += distance
            elif axis == "y":
                package.position.y += distance
            else:
                package.position.z += distance

        elif operation == "rotate":
            axis = rng.choice(["x", "y", "z"])
            direction = rng.choice([RotationDirection.POSITIVE,
RotationDirection.NEGATIVE])
            if axis == "x":
                package.rotate_x(direction)
            elif axis == "y":
                package.rotate_y(direction)
            else:
                package.rotate_z(direction)

        else:
            package.move(None)

    mutated.rebuild()

    return mutated
```

Pembentukan Generasi Berikutnya

Fungsi create_next_generation mengulangi pemilihan dua induk, crossover, mutasi, dan evaluasi sampai jumlah anak sama dengan ukuran populasi. Seluruh populasi lama diganti anak. Program tidak menyisipkan elite secara otomatis.

```
def create_next_generation(evaluated_population, mutation_rate,
rng):
    if not evaluated_population:
        raise ValueError("Populasi tidak boleh kosong")

    next_population = []
```

```
    population_size = len(evaluated_population)

    while len(next_population) < population_size:
        parent_1 = roulette_selection(evaluated_population, rng)
        parent_2 = roulette_selection(evaluated_population, rng)
        child = crossover(parent_1, parent_2, rng)
        child = mutate(child, mutation_rate, rng)
        next_population.append({
            "state": child,
            "evaluation": evaluate_state(child)
        })

    return next_population
```

Fungsi update_best_valid memeriksa individu yang valid dan menyimpan salinan dengan value tertinggi. Arsip ini tidak dimasukkan kembali ke populasi. Fungsinya menjaga hasil valid tetap tersedia meskipun individu tersebut hilang pada generasi berikutnya.

```
def update_best_valid(best_valid, evaluated_population):
   for individual in evaluated_population:
      evaluation = individual["evaluation"]
      if not evaluation["is_valid"]:
         continue
      if best_valid is None or evaluation["value"] > best_valid["evaluation"]["value"]:
         best_valid = deepcopy(individual)

   return best_valid
```

Pengendalian Proses Genetic Algorithm

Fungsi run_ga menerima Problem dan parameter populasi, jumlah generasi, peluang mutasi, seed, serta peluang muat awal. Program membuat populasi, mengevaluasinya, menyimpan salinan awal, dan memperbarui arsip valid. Generasi nol dicatat sebelum reproduksi dimulai.

Generator populasi menggunakan seed yang diberikan. Keputusan evolusi memakai generator lain dengan seed yang ditambah satu. Setiap generasi memperbarui populasi, arsip valid, dan history. Setelah loop selesai, program menyimpan individu terbaik pada populasi terakhir dan durasi pencarian.

```
def run_ga(problem, population_size=20, generation_count=100,
mutation_rate=0.2, seed=None, load_probability=0.5):
    if type(generation_count) is not int or generation_count < 0:
        raise ValueError("Generation count harus berupa bilangan
bulat nonnegatif")

    if type(mutation_rate) not in (int, float):
        raise ValueError("Mutation rate harus berupa angka antara 0
dan 1")

    if not 0 <= mutation_rate <= 1:
        raise ValueError("Mutation rate harus berada antara 0 dan
1")

    started = perf_counter()
    population = generate_population(problem,
population_size=population_size, seed=seed,
load_probability=load_probability)

    evaluated = evaluate_population(population)
    initial_population = deepcopy(evaluated)
    best_valid = update_best_valid(None, evaluated)
    history = [summarize_generation(evaluated, 0, best_valid)]

    evolution_seed = None

    if seed is not None:
        evolution_seed = seed + 1

    rng = Random(evolution_seed)

    for generation in range(1, generation_count + 1):
        evaluated = create_next_generation(evaluated,
mutation_rate, rng)
        best_valid = update_best_valid(best_valid, evaluated)
        history.append(summarize_generation(evaluated, generation,
best_valid))

    best_individual = deepcopy(find_best_individual(evaluated))
    duration = perf_counter() - started
```

Data Hasil dan Riwayat Populasi

Fungsi summarize_generation menghitung maksimum dan rerata fitness, maksimum dan rerata value, jumlah individu valid, serta value arsip valid. Perhitungannya melibatkan seluruh individu populasi, termasuk yang tidak valid.

Untuk populasi berukuran N pada generasi t, pencatatan value dilakukan sebagai berikut.

$$V_{maks}(t) = max(V_1(t), \ldots, V_N(t)) \quad (17)$$

$$\overline{V}(t) = \frac{\sum_{i=1}^{N} V_i(t)}{N} \quad (18)$$

Field hasil	Isi
initial_population	Seluruh individu awal beserta evaluasinya.
final_population	Seluruh individu pada generasi terakhir.
best_individual	Individu dengan fitness tertinggi pada populasi terakhir.
best_valid	Arsip individu valid dengan value tertinggi sepanjang pencarian.
history	Catatan generasi nol sampai generasi terakhir.
duration_seconds	Durasi pencarian sebelum visualisasi.
parameters	Pengaturan yang digunakan selama run.

Arsip best_valid dapat bernilai None jika belum ada solusi valid. Best_individual tetap tersedia walaupun tidak valid. Keduanya tidak boleh dianggap sebagai hasil yang sama. Banyak generasi yang lebih besar juga tidak menjamin fitness akhir meningkat.

Penyajian Populasi Genetic Algorithm

Fungsi visualize_genetic_algorithm memilih individu awal dengan fitness tertinggi. Tampilan akhir memakai arsip best_valid jika tersedia, atau best_individual dari populasi terakhir jika belum ada solusi valid. Jadi, gambar akhir dapat berasal dari generasi sebelumnya.

```
final_individual = result["best_valid"]

 if final_individual is None:
     final_individual = result["best_individual"]


 initial_state = initial_individual["state"]
 final_state = final_individual["state"]
```

```
    initial_value = initial_individual["evaluation"]["state_value"]
    final_value = final_individual["evaluation"]["state_value"]
```

Grafik membaca max_fitness dan mean_fitness sehingga menunjukkan skor setelah penalti. Field max_value dan mean_value juga tersedia untuk menggambarkan objective function sebelum penalti. Kedua besaran perlu dibedakan saat menyajikan hasil eksperimen.

```
   generations = []
    maximum = []
    average = []

    for generation in result["history"]:
        generations.append(generation["generation"])
        maximum.append(generation["max_fitness"])
        average.append(generation["mean_fitness"])

    figure, initial_ax, final_ax, graph_ax, info_ax =
create_figure()

    draw_state(initial_ax, initial_state, "Best Initial
Individual", initial_value)
    draw_state(final_ax, final_state, "Best Final Individual",
final_value)

    graph_ax.plot(generations, maximum, label="Maximum")
    graph_ax.plot(generations, average, label="Average")
    graph_ax.set(title="Fitness terhadap Generasi",
xlabel="Generasi", ylabel="Fitness")
    graph_ax.legend()
    graph_ax.grid(alpha=0.3)
```

Test Case

Rancangan Pengujian

Pengujian genetic algorithm memakai berkas tests/medium.json yang berisi 30 paket dengan truk berukuran 10 x 10 x 8 dan kapasitas 90 unit berat. Semua paket awalnya berada di luar truk. Objective function yang dipakai adalah state_value

tanpa ETA, yaitu total value paket yang dimuat dikurangi penalti. Mutation rate dibuat tetap 0,2 dan peluang muat awal 0,5.

Terdapat dua kelompok eksperimen. Kelompok pertama menjadikan jumlah populasi sebagai kontrol (20 individu) dengan banyak iterasi 25, 50, dan 100 generasi. Kelompok kedua menjadikan banyak iterasi sebagai kontrol (50 generasi) dengan jumlah populasi 10, 20, dan 40 individu. Setiap konfigurasi dijalankan tiga kali dengan seed berbeda yang dihitung dari 1000 x populasi + 10 x generasi + nomor run sehingga hasilnya dapat diulang. Konfigurasi 20 individu dan 50 generasi berada pada kedua kelompok sehingga ketiga run-nya dipakai bersama.

Data yang dicatat untuk setiap run adalah fitness individu terbaik populasi awal, fitness individu terbaik populasi akhir, fitness tertinggi sepanjang generasi, rata-rata fitness populasi akhir, ada tidaknya solusi valid, dan durasi pencarian. State awal dan akhir beserta grafik fitness maksimum dan rata-rata terhadap generasi disimpan sebagai gambar pada folder output

Tabel berikut berisi hasil seluruh run genetic algorithm.

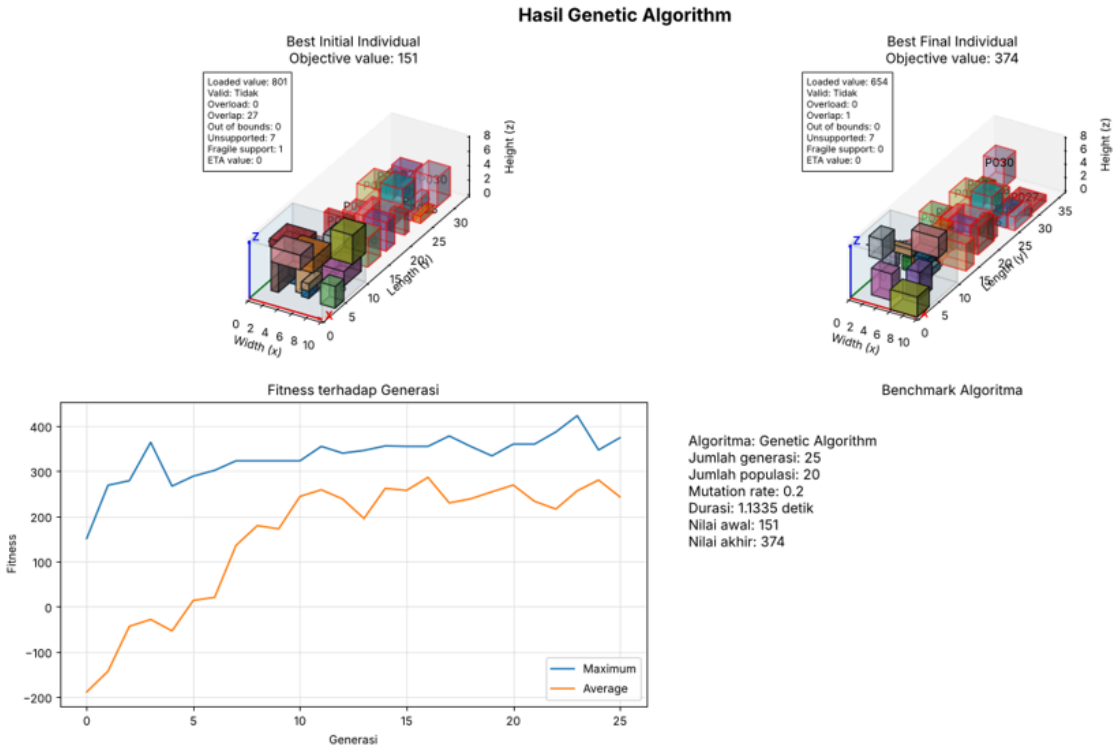
Kelompok	Populasi	Generasi	Run	Fitness awal	Fitness akhir	Fitness tertinggi	Rata-rata akhir	Valid	Durasi (s)
Iterasi	20	25	1	151	374	423	243.3	Tidak	1.13
Iterasi	20	25	2	150	538	581	430.2	Tidak	1.28
Iterasi	20	25	3	224	526	536	438.6	Tidak	1.33
Iterasi	20	50	1	235	575	589	437.1	Tidak	2.6

Iterasi	20	50	2	333	635	650	523.5	Tidak	2.79
Iterasi	20	50	3	230	522	532	364.4	Tidak	2.47
Iterasi	20	100	1	367	559	638	493.6	Tidak	6.09
Iterasi	20	100	2	284	690	719	581.6	Tidak	5.45
Iterasi	20	100	3	317	688	698	554.2	Tidak	5.15
Populasi	10	50	1	93	389	445	344.2	Tidak	1.56
Populasi	10	50	2	160	320	413	267.5	Tidak	1.21
Populasi	10	50	3	39	471	520	366.0	Tidak	1.47
Populasi	20	50	1	235	575	589	437.1	Tidak	2.57
Populasi	20	50	2	333	635	650	523.5	Tidak	2.84
Populasi	20	50	3	230	522	532	364.4	Tidak	2.86
Populasi	40	50	1	201	691	691	510.3	Tidak	5.3

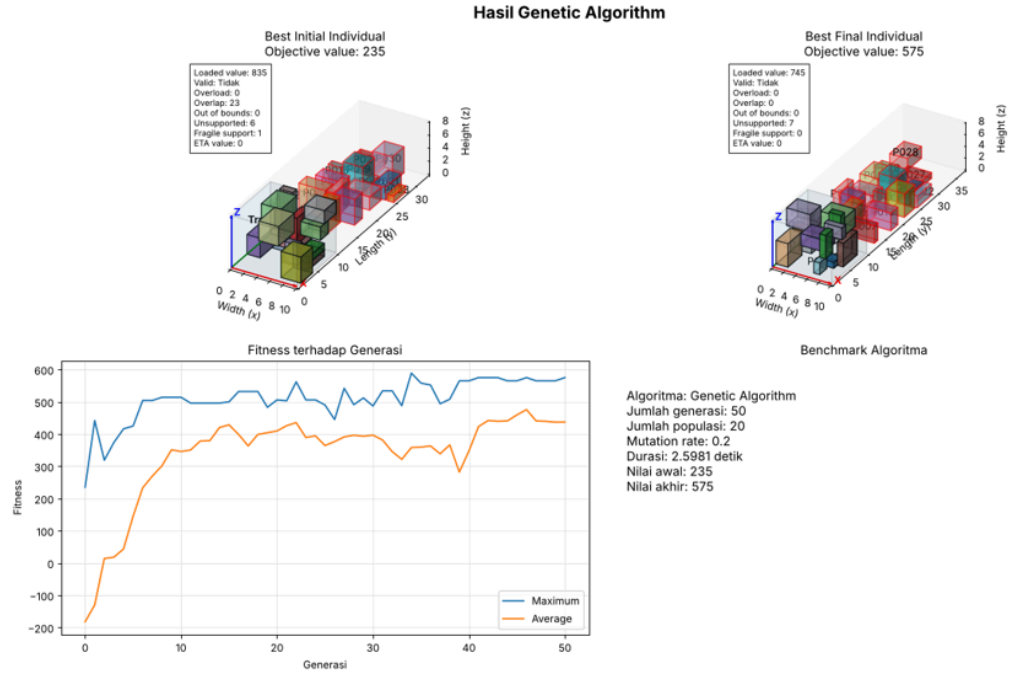
Populasi	40	50	2	317	630	721	454.8	Tidak	5.77
Populasi	40	50	3	215	545	601	412.9	Tidak	5.46

Ringkasan per konfigurasi (fitness akhir dari tiga run):

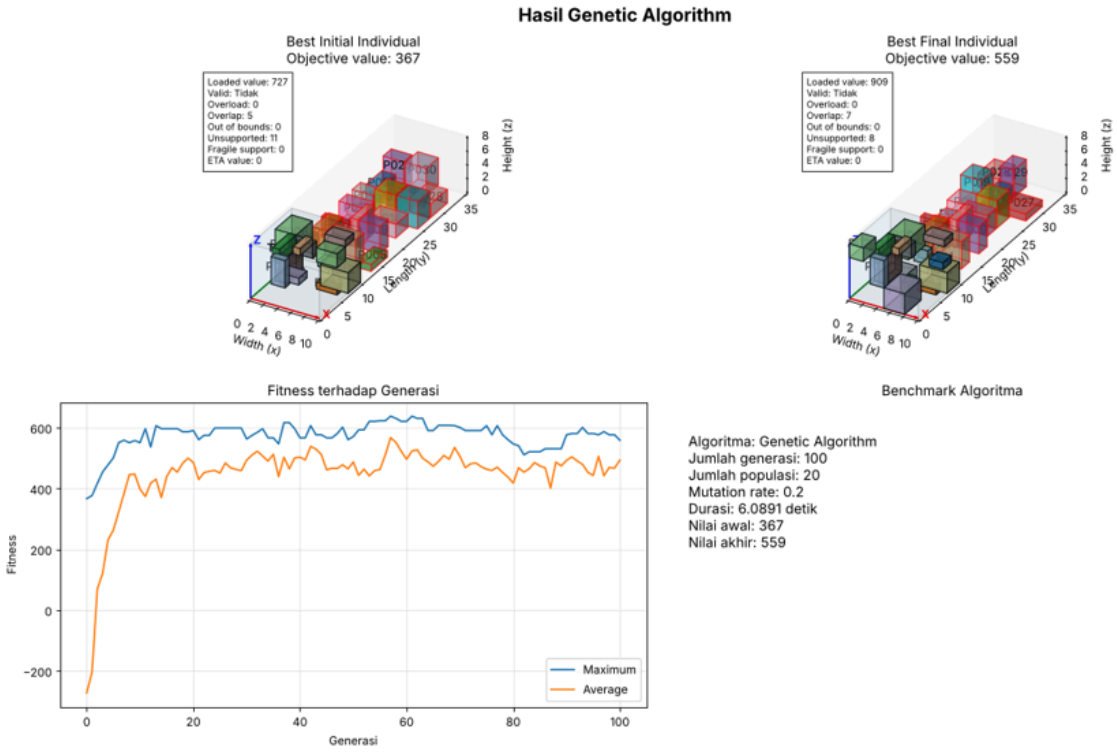
Konfigurasi	Rata-rata	Simpangan baku	Minimum	Maksimum	Durasi rata-rata (s)
Generasi = 25	479.3	74.6	374	538	1.25
Generasi = 50	577.3	46.2	522	635	2.62
Generasi = 100	645.7	61.3	559	690	5.56
Populasi = 10	393.3	61.7	320	471	1.41
Populasi = 20	577.3	46.2	522	635	2.76
Populasi = 40	622	59.9	545	691	5.51



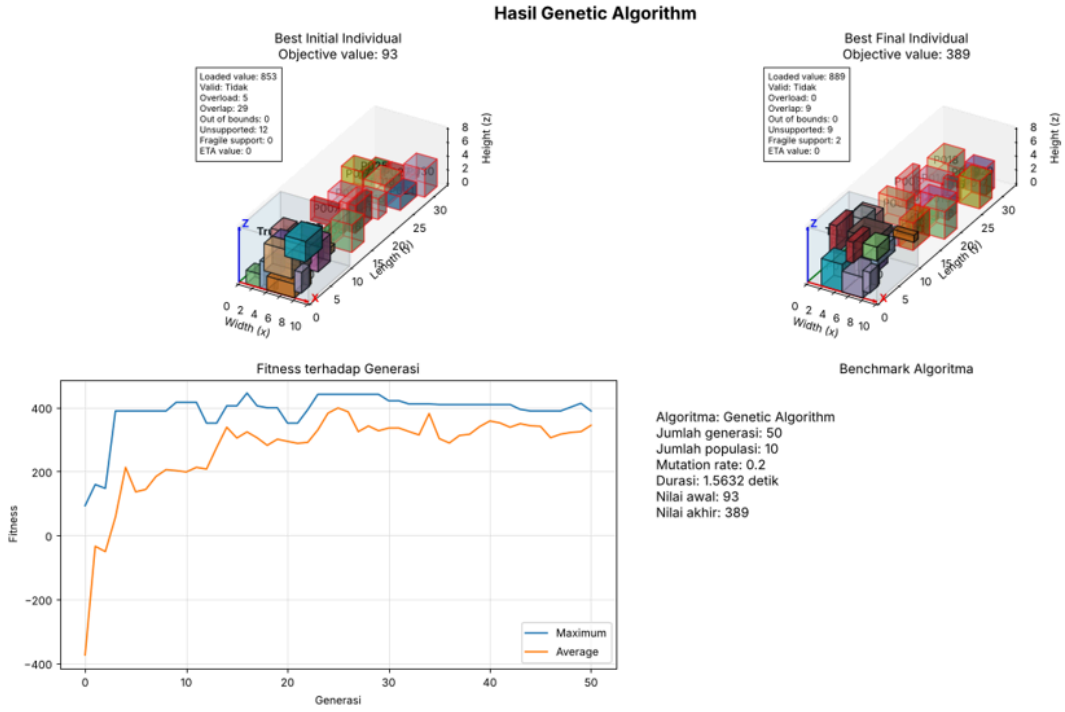
Gambar hasil GA dengan populasi 20 dan 25 generasi (run 1).



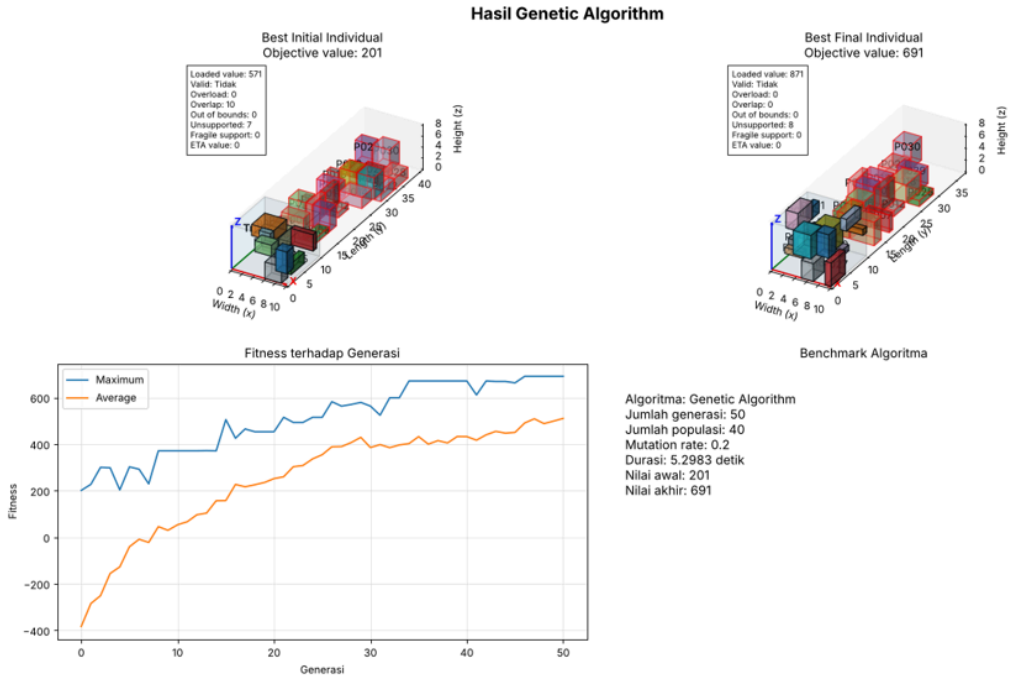
Gambar hasil GA dengan populasi 20 dan 50 generasi (run 1).



Gambar hasil GA dengan populasi 20 dan 100 generasi (run 1).



Gambar hasil GA dengan populasi 10 dan 50 generasi (run 1).



Gambar hasil GA dengan populasi 40 dan 50 generasi (run 1).

Analisis Hasil

Pengaruh banyak iterasi. Dengan populasi 20, rata-rata fitness akhir naik dari 479,3 pada 25 generasi menjadi 577,3 pada 50 generasi dan 645,7 pada 100 generasi. Generasi tambahan memberi lebih banyak kesempatan bagi seleksi, crossover, dan mutasi untuk memasukkan paket bernilai tinggi dan mengurangi pelanggaran. Kenaikannya mengecil pada rentang 50 ke 100 generasi (68,4) dibandingkan 25 ke 50 generasi (98,0), sedangkan durasi naik hampir linear dari 1,25 detik menjadi 2,62 dan 5,56 detik.

Pengaruh jumlah populasi. Dengan 50 generasi, rata-rata fitness akhir naik dari 393,3 pada populasi 10 menjadi 577,3 pada populasi 20 dan 622,0 pada populasi 40. Populasi yang lebih besar menyimpan variasi susunan yang lebih banyak sehingga roulette selection mempunyai lebih banyak induk yang baik. Populasi 10 terlihat

paling lemah karena keragaman cepat hilang. Durasi juga naik sebanding dengan jumlah individu karena setiap generasi mengevaluasi seluruh populasi.

Konsistensi. Simpangan baku fitness akhir berada pada rentang 46,2 sampai 74,6 untuk semua konfigurasi. Selisih antar-run masih cukup besar karena initial state dan operator genetik bersifat acak. Pada beberapa run, fitness akhir lebih rendah dari fitness tertinggi yang pernah dicapai, misalnya populasi 40 run 2 (630 dibanding 721) dan populasi 20 dengan 100 generasi run 1 (559 dibanding 638). Hal ini terjadi karena implementasi tidak memakai elitism sehingga individu terbaik dapat hilang pada generasi berikutnya.

Kedekatan dengan global optimum. Tidak ada run yang menghasilkan solusi valid, sehingga arsip best_valid selalu bernilai None. Fitness yang positif berasal dari total value yang lebih besar daripada penalti, bukan dari susunan tanpa pelanggaran. Crossover satu titik menggabungkan posisi paket dari dua induk tanpa memeriksa irisan, sehingga anak sering mengandung overlap atau paket yang melayang. Mutasi juga hanya menggeser paket satu sel atau merotasinya. Akibatnya, genetic algorithm pada dataset ini belum mendekati global optimum yang valid. Solusi valid lebih mungkin diperoleh dengan elitism, penalti yang lebih besar dibandingkan value paket, atau perbaikan anak (repair) setelah crossover.

Secara keseluruhan, menambah generasi dan populasi meningkatkan fitness akhir dengan konsekuensi waktu yang bertambah sebanding. Pada eksperimen ini, populasi 20 dengan 100 generasi dan populasi 40 dengan 50 generasi memberi hasil terbaik dengan durasi yang hampir sama, yaitu sekitar 5,5 detik.

Kesimpulan dan Saran

Ketiga algoritma memakai model paket, koordinat, grid, dan evaluator yang sama. State awal dan pembaruan kandidat berbeda sesuai metode pencarian. Hill climbing memilih perpindahan berdasarkan perbandingan fitness, simulated annealing menambahkan peluang menerima kandidat yang lebih buruk, dan genetic algorithm membentuk populasi baru melalui seleksi, crossover, serta mutasi.

Pemisahan value, fitness, dan validitas diperlukan agar hasil pencarian dapat dijelaskan dengan tepat. Data hasil program sudah menyediakan state dan catatan proses yang dibutuhkan untuk pembahasan berikutnya. Sebelum menyajikan gambar eksperimen, field yang dibaca visualisasi perlu diperiksa agar sesuai dengan besaran yang diberi label.

Pembagian Tugas Kelompok

No	Nama / NIM Anggota	Pembagian Tugas
1	Muhammad Ashkar / 13524007	Implementasi objective function, Implementasi simulated annealing, bonus ETA, bonus multi-truck
2	Fayyaz Akmal Lauda / 13524076	Implementasi handle input, case, initial state, genetic algorithm
3	Muhammad Faiz Alfada Dharma / 13524097	Implementasi semua algoritma hill climbing, program utama, output, dan visualisasi

Format Deklarasi Penggunaan AI

Surat Pernyataan Penggunaan AI

Dengan ini, kami:

Nama/NIM 1 : Muhammad Ashkar / 13524007

Nama/NIM 2 : Fayyaz Akmal Lauda / 13524076

Nama/NIM 3 : Muhammad Faiz Alfada Dharma / 13524097

Fakultas/Sekolah : STEI-K

Kode Mata Kuliah : IF3170

Nama Mata Kuliah : Intelegensi Artifisial

Judul Tugas/Proposal : Tugas Besar 1 IF3170 Intelegensi Artifisial

Menyatakan bahwa kami **menggunakan** AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Menggunakan LLM untuk mengecek apakah terdapat syntax error atau error kode yang seharusnya terjadi saat compile time.
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Menggunakan LLM untuk memperbanyak file test case yang bisa digunakan.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti	Ya	Menggunakan LLM untuk mencari referensi struktur

ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.		folder yang digunakan dan untuk belajar cara menggunakan library dan syntax python.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:	Tidak	

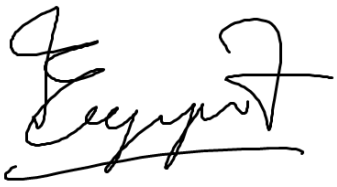
Kam* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 19 Oktober 2026



Muhammad Ashkar

13524007



Fayyaz Akmal Lauda

13524076



Muhammad Faiz Alfada D

13524097

Referensi

- Tim Pengampu IF3170. Beyond Classical Search. Materi kuliah Semester I tahun ajaran 2026/2027.
- https://www.sciencedirect.com/topics/computer-science/cooling-schedule